

Strukture podataka i algoritmi

Python (1. deo)

dr Danijel Čabarkapa

Akademija strukovnih studija Šabac

Odsek za medicinske i poslovno-tehnološke studije

Sadržaj:

1. Karakteristike jezika Python
2. Numerički podaci i promenljive
3. Python strukture podataka

1. Karakteristike jezika Python

Python

- Python je kreirao Guido van Rossum početkom '90-ih godina XX veka u National Research Institute for Mathematics and Computer Science, Holandija
- Python je nastao na osnovu jezika ABC, Modul 3, C, C++, Algol 68, SmallTalk, Unix shell itd.
- Python je viši programski, interpreterski, interaktivni, objektno-orijentisani jezik
- Python je interaktivan (kôd se može pisati direktno u Python promptu) i dizajniran je da bude vrlo čitljiv, tako da ima specifičan način pisanja kôda
- Python se procesira u vreme izvršavanja od strane interpretera (program ne mora da se prevodi da bi se zatim izvršio)

Python karakteristike 1

Python karakteristike:

- Lak za učenje (malo ključnih reči, jednostavna struktura i „čista“ sintaksa)
- Lak za čitanje (Python kôd je predviđen da bude jasno definisan i veoma čitljiv)
- Koristi standardne biblioteke, tako da je kompatibilan za razne platforme: UNIX, Windows, Mac, Linux
- Posедуje interaktivni mod rada
- Prenosivost: Python se može pokrenuti sa istim interfejsom na različitim hardverskim platformama
- Proširivost različitim modulima niskog nivoa čime se može postići da alat koji se koristi bude što efikasniji

Python karakteristike 2

Python karakteristike:

- Baze podataka: Python omogućava interfejse ka svim značajnim komercijalnim bazama podataka
- GUI programiranje (grafički interfejsi)
- Python je objektno-orijentisan jezik pa je omogućeno lakše održavanje kompleksnih programski kôdova
- Skalabilnost: Python obezbeđuje dobru strukturu i podršku za velike programe
- Python podržava metode funkcionalnog, strukturnog i OOP principa programiranja
- Može se koristiti kao skript jezik ili za izradu velikih aplikacija
- Python podržava automatsku dealokaciju (*garbage collection*)

Python-TIOBE lista

Na adresi <https://www.tiobe.com/tiobe-index/> može se videti lista popularnosti programskih jezika za 2024. i 2025. godinu, Python je u poslednje 2 godine na prvom mestu

Apr 2025	Apr 2024	Change	Programming Language		Ratings	Change
1	1			Python	23.08%	+6.67%
2	3	▲		C++	10.33%	+0.56%
3	2	▼		C	9.94%	-0.27%
4	4			Java	9.63%	+0.69%
5	5			C#	4.39%	-2.37%
6	6			JavaScript	3.71%	+0.82%
7	7			Go	3.02%	+1.17%
8	8			Visual Basic	2.94%	+1.24%
9	11	▲		Delphi/Object Pascal	2.53%	+1.06%
10	9	▼		SQL	2.19%	+0.57%

Python primene

- Primena Python programskog jezika:
 - ▶ Shell tools, System admin tools, Command line programs
 - ▶ Web development (backend razvoj web aplikacija npr. uz pomoć Django ili Flask frameworka)
 - ▶ Mašinsko učenje (sa alatima Scikit-learn, TensorFlow i Keras za izradu i treniranje modela mašinskog učenja)
 - ▶ Analiza podataka (Data Science i analiza podataka pomoću Pandas, NumPy i Matplotlib biblioteka koje olakšavaju manipulaciju, analizu i vizualizaciju podataka)
 - ▶ Automatizacija i skriptovanje (Python se koristi za automatizaciju zadataka i pisanje skripti koje pojednostavljaju složene procese i povećavaju produktivnost)
 - ▶ Naučni proračuni (pomoću biblioteka SciPy ili SymPy, Python se često koristi u naučnim istraživanjima i inženjeringu za rešavanje složenih matematičkih problema)

Python interpreter

- Program napisan u višem programskom jeziku predstavlja izvorni kôd
- Izvorni kôd se mora prevesti u mašinski jezik, a to se može izvršiti prevođenjem ili interpretiranjem
- **Prevodilac** direktno prevodi izvorni kôd u mašinski, objektni kôd i može se pokretati i izvršavati na ciljnoj arhitekturi
- **Interpreter** je program koji izvršava naredbe programskog jezika korišćenjem svog skupa instrukcija
 - ▶ Program se izvršava naredbu po naredbu koje se prevode u trenutku izvršavanja
 - ▶ Dodatni nivo apstrakcije
- Prednost korišćenja interpretera su:
 - ▶ Brži razvoj i testiranje programa (nema potrebe za stalnim prevođenjem programa u mašinski oblik)
 - ▶ Lakše utvrđivanje grešaka (čim se pojavi greška, interpreter staje sa radom)

Python interpreter

- Nedostatak interpretera su lošije performanse izvršavanja zbog postojanja dodatnog sloja a to se ispravlja prevođenjem produkcijske verzije kôda ili **JIT** (*Just In Time*) prevođenjem u modernim sistemima
- Python interpreter izvršava:
 - ▶ Direktno zadate naredbe u interaktivnom režimu rada
 - ▶ Napisane skripte i module sačuvane u datoteci
- **Skripta** je datoteka koja sadrži sekvencu naredbi namenjenih za direktno izvršavanje, dok je **modul** deo kôda namenjen za uvoz u druge Python datoteke ili skripte
- Python interpreter izvršava skriptu u tri koraka:
 - 1) Redom prođe kroz skriptu kako bi utvrdio sintaksnu korektnost
 - 2) Prevede izvorni kôd u međukôd (*bytecode*) kako bi se omogućilo sprovođenje optimizacija
 - 3) Pošalje kôd na izvršavanje izvršnom okruženju PVM (*Python Virtual Machine*) i izvršava kôd naredbu po naredbu

Python interpreteri

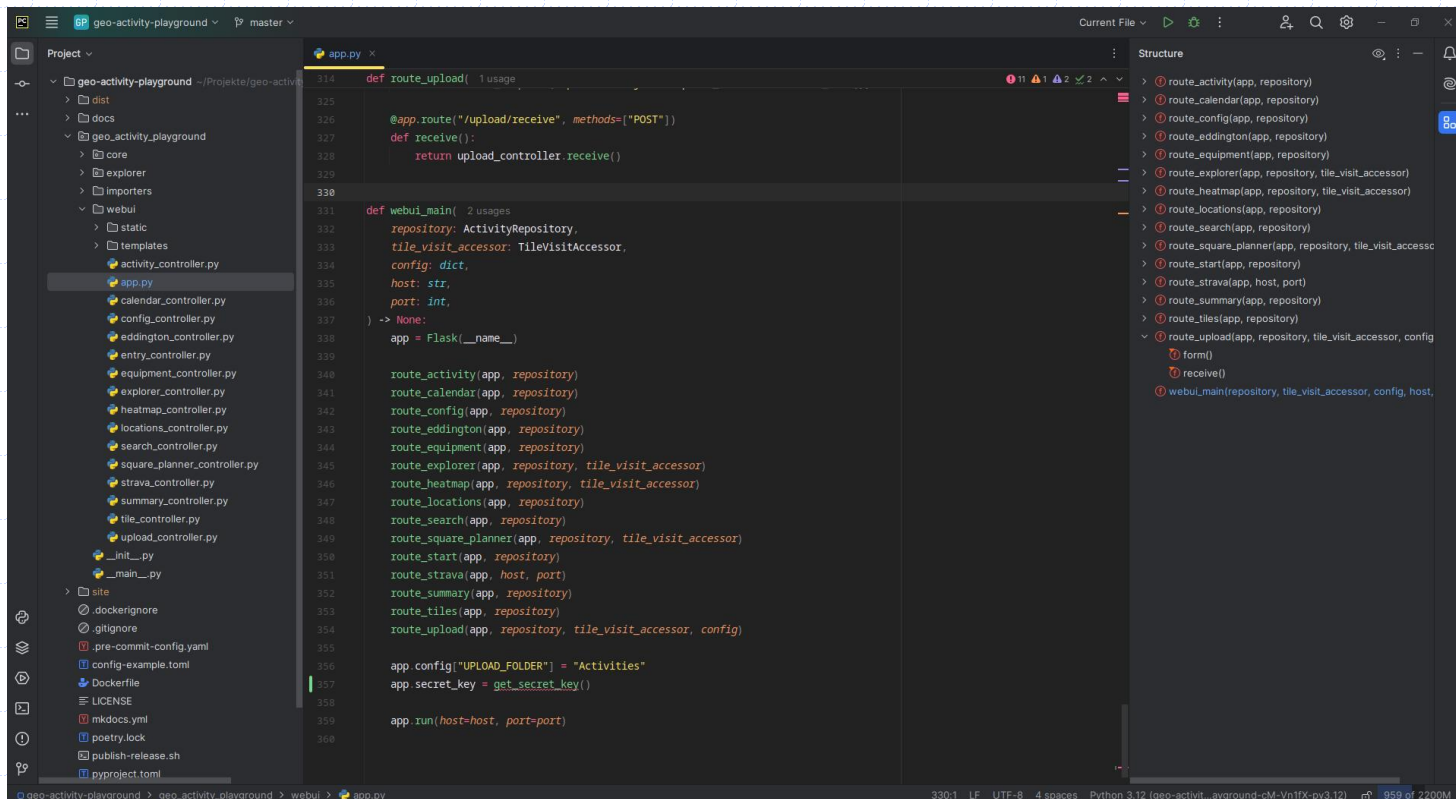
- 1) **Python** ili **CPython** (napisan u jeziku C/C++)
- 2) **PyPy** (koristi JIT kompajler koji prevodi u mašinski kôd u hodu, što omogućava visok nivo performansi pa je nekoliko puta brži od CPython verzije)
- 3) **IronPython** (napisan u jeziku C# za .NET okruženje, poslednja verzija 3.4.2)
- 4) **MicroPython** (Python optimizovan za rad sa mikrokontrolerima, pokreće se u pretraživaču)
- 5) **Stackless Python**, verzija CPython koja ne koristi sopstveni stek i podržava mikronitni način rada (microthreads), čime se dodaje podrška za korutine, komunikaciju između kanala i serijalizacija zadataka
- 6) **Jython** (napisan u jeziku Java i izvršava se u JVM-Java Virtual Machine okruženju)

Python razvojna okruženja

- Integrisano razvojno okruženje IDE (Integrated Development Environment) čini skup alata koji se koriste prilikom razvoja programa
- IDE sadrži editor teksta, debugger, prevodilac ili interpreter i sl.
- Najpoznatija razvojna okruženja za Python:
 1. PyCharm (na vežbama!)
 2. IDLE (na vežbama!)
 3. Visual Studio Code (na vežbama!!!)
 4. Atom
 5. Sublime Text
 6. Spyder
 7. PyDev (za Eclipse)
 8. Jupyter

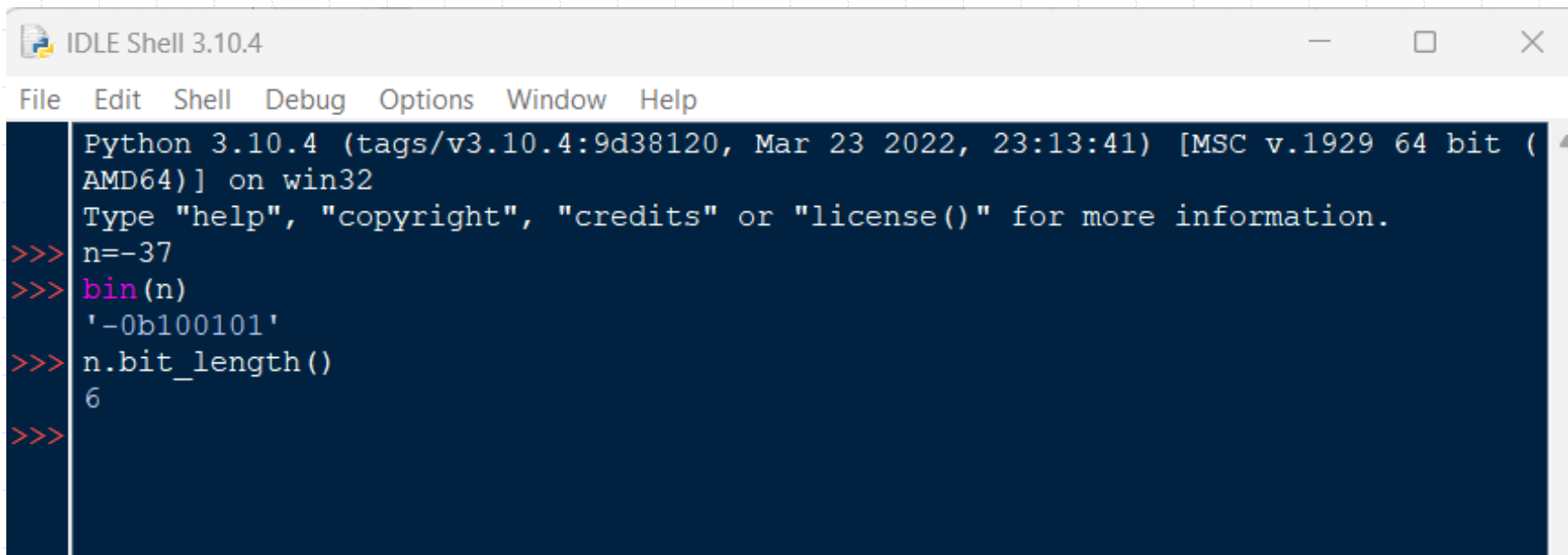
PyCharm

IDE za razvoj aplikacija u Pythonu i Javi, sadrži analizator koda, grafički debugger, integrisane test framework-e (pytest, nose, doctest, tox), sisteme za verzioniranje (Git, Github, Mercurial, Perforce, Subversion), integracija sa SQL bazama podataka, proizvod poznate kompanije JetBrains, zvanična internet stranica <https://www.jetbrains.com/pycharm/>



IDLE

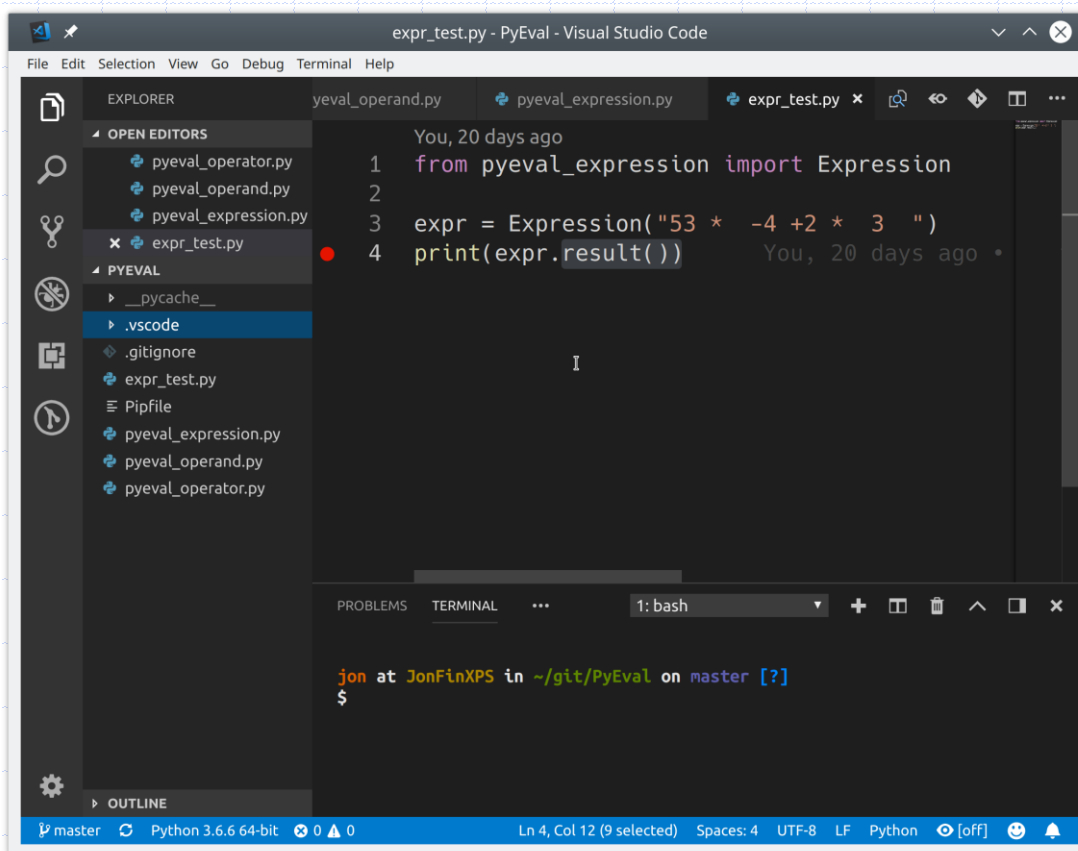
IDLE (Integrated DeveLopment Environment for Python) je besplatno okruženje koje dolazi uz Python instalaciju, namenjen za interaktivan rad u konzoli, višeplatformska 100% čista Python aplikacija zasnovana na Tkinter GUI, uključuje editor teksta optimizovan za editovanje Python izvornog koda, integrisani interaktivni program za otklanjanje grešaka i nekoliko specijalizovanih web browsera

A screenshot of the IDLE Shell 3.10.4 window. The window has a title bar with the text 'IDLE Shell 3.10.4' and standard window controls (minimize, maximize, close). Below the title bar is a menu bar with the following items: File, Edit, Shell, Debug, Options, Window, and Help. The main area of the window is a dark blue console with white text. It shows the Python 3.10.4 startup message: 'Python 3.10.4 (tags/v3.10.4:9d38120, Mar 23 2022, 23:13:41) [MSC v.1929 64 bit (AMD64)] on win32'. Below this, it says 'Type "help", "copyright", "credits" or "license()" for more information.' The user has entered several commands: '>>> n=-37', '>>> bin(n)', and '>>> n.bit_length()'. The output for 'bin(n)' is '-0b100101' and for 'n.bit_length()' is '6'. The prompt '>>>' is shown at the end of the last line.

```
Python 3.10.4 (tags/v3.10.4:9d38120, Mar 23 2022, 23:13:41) [MSC v.1929 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>> n=-37
>>> bin(n)
'-0b100101'
>>> n.bit_length()
6
>>>
```

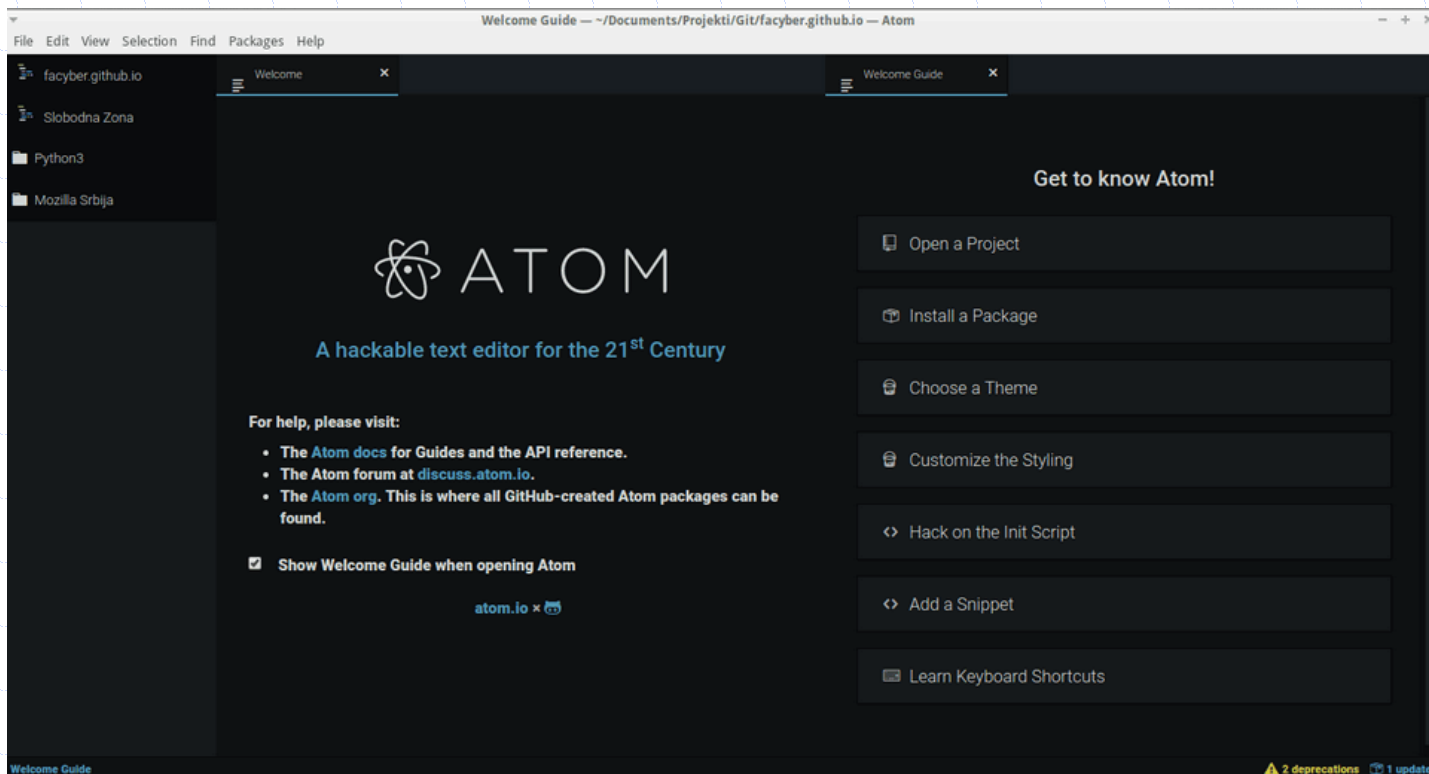
Visual Studio Code (VSC)

VSC je Microsoftov IDE koji je besplatan, otvorenog koda i kompatibilan za Windows, Linux i macOS, poseduje brojne napredne funkcije i ekstenzije koje se stalno ažuriraju, dolazi sa ugrađenom podrškom za JavaScript, Node.js i TypeScript, a za druge programske jezike postoje ekstenzija kao npr. za Python, C#, PHP, Rust i stotinu drugih, zvanični sajt je <https://code.visualstudio.com/>



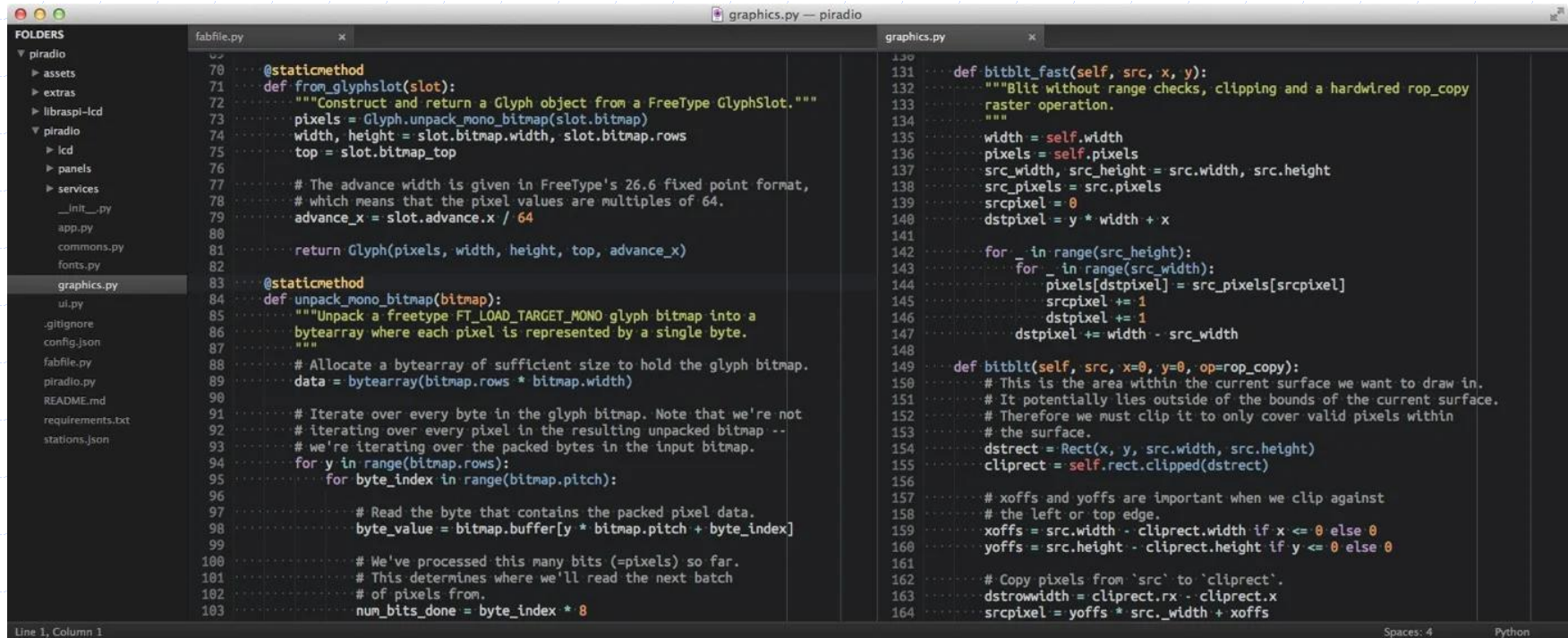
Atom

Atom je open-source hackable IDE koji je razvio GitHub, što znači da se mogu menjati delovi kôda kako bi se nešto značajno promenilo u samom programu bez diranja „core“ fajlova, pisanjem/dodavanjem kôda mogu se menjati podešavanja programa, fontovi, teme, slično programiranju web strane (jer je Atom pisan u HTML, CSS i JavaScript jezicima), Atom je brz i stabilan, lak za korišćenje i podešavanje, zvanična stranica je <https://atom-editor.cc/>



Sublime Text

Sublime Text je komercijalni IDE koji se smatra jednim od najboljih, pisan je u Pythonu, uz pomoć velikog broja ekstenzija moguće je drastično proširiti osnovne mogućnosti editora i izvršiti posebno podešavanje za kodiranje u nekom od jezika a preduslov za to je instaliranje Package Control paketa, podržan za sve značajnije operative sisteme, zvanična stranica je <https://www.sublimetext.com/>



```
70 @staticmethod
71 def from_glyphslot(slot):
72     """Construct and return a Glyph object from a FreeType GlyphSlot."""
73     pixels = Glyph.unpack_mono_bitmap(slot.bitmap)
74     width, height = slot.bitmap.width, slot.bitmap.rows
75     top = slot.bitmap_top
76
77     # The advance width is given in FreeType's 26.6 fixed point format,
78     # which means that the pixel values are multiples of 64.
79     advance_x = slot.advance_x / 64
80
81     return Glyph(pixels, width, height, top, advance_x)
82
83 @staticmethod
84 def unpack_mono_bitmap(bitmap):
85     """Unpack a freetype FT_LOAD_TARGET_MONO glyph bitmap into a
86     bytearray where each pixel is represented by a single byte.
87     """
88     # Allocate a bytearray of sufficient size to hold the glyph bitmap.
89     data = bytearray(bitmap.rows * bitmap.width)
90
91     # Iterate over every byte in the glyph bitmap. Note that we're not
92     # iterating over every pixel in the resulting unpacked bitmap --
93     # we're iterating over the packed bytes in the input bitmap.
94     for y in range(bitmap.rows):
95         for byte_index in range(bitmap.pitch):
96
97             # Read the byte that contains the packed pixel data.
98             byte_value = bitmap.buffer[y * bitmap.pitch + byte_index]
99
100             # We've processed this many bits (=pixels) so far.
101             # This determines where we'll read the next batch
102             # of pixels from.
103             num_bits_done = byte_index * 8
```

```
130
131 def bitblt_fast(self, src, x, y):
132     """Blit without range checks, clipping and a hardcoded rop_copy
133     raster operation.
134     """
135     width = self.width
136     pixels = self.pixels
137     src_width, src_height = src.width, src.height
138     src_pixels = src.pixels
139     srcpixel = 0
140     dstpixel = y * width + x
141
142     for _ in range(src_height):
143         for _ in range(src_width):
144             pixels[dstpixel] = src_pixels[srcpixel]
145             srcpixel += 1
146             dstpixel += 1
147         dstpixel += width - src_width
148
149 def bitblt(self, src, x=0, y=0, op=rop_copy):
150     """This is the area within the current surface we want to draw in.
151     It potentially lies outside of the bounds of the current surface.
152     Therefore we must clip it to only cover valid pixels within
153     the surface.
154     dstrect = Rect(x, y, src.width, src.height)
155     cliprect = self.rect.clipped(dstrect)
156
157     # xoffs and yoffs are important when we clip against
158     # the left or top edge.
159     xoffs = src.width - cliprect.width if x <= 0 else 0
160     yoffs = src.height - cliprect.height if y <= 0 else 0
161
162     # Copy pixels from 'src' to 'cliprect'.
163     dstrowwidth = cliprect.rx - cliprect.x
164     srcpixel = yoffs * src.width + xoffs
```

Spyder

Spyder je IDE otvorenog koda (pod licencom MIT) namenjen za naučno programiranje u Pythonu, jer integriše NumPy, SciPy, Matplotlib, IPython i druge biblioteke uz proširivost sa dodacima, podrška za interaktivne alate za inspekciju podataka, kontrole kvaliteta specifične za Python preko alata Pyflakes, Pylint i Rope, Spyder je cross-platform IDE (Anaconda) i dostupan je za Windows, Linux i macOS, zvanični sajt je <https://www.spyder-ide.org/>

The screenshot displays the Spyder IDE interface with the following components:

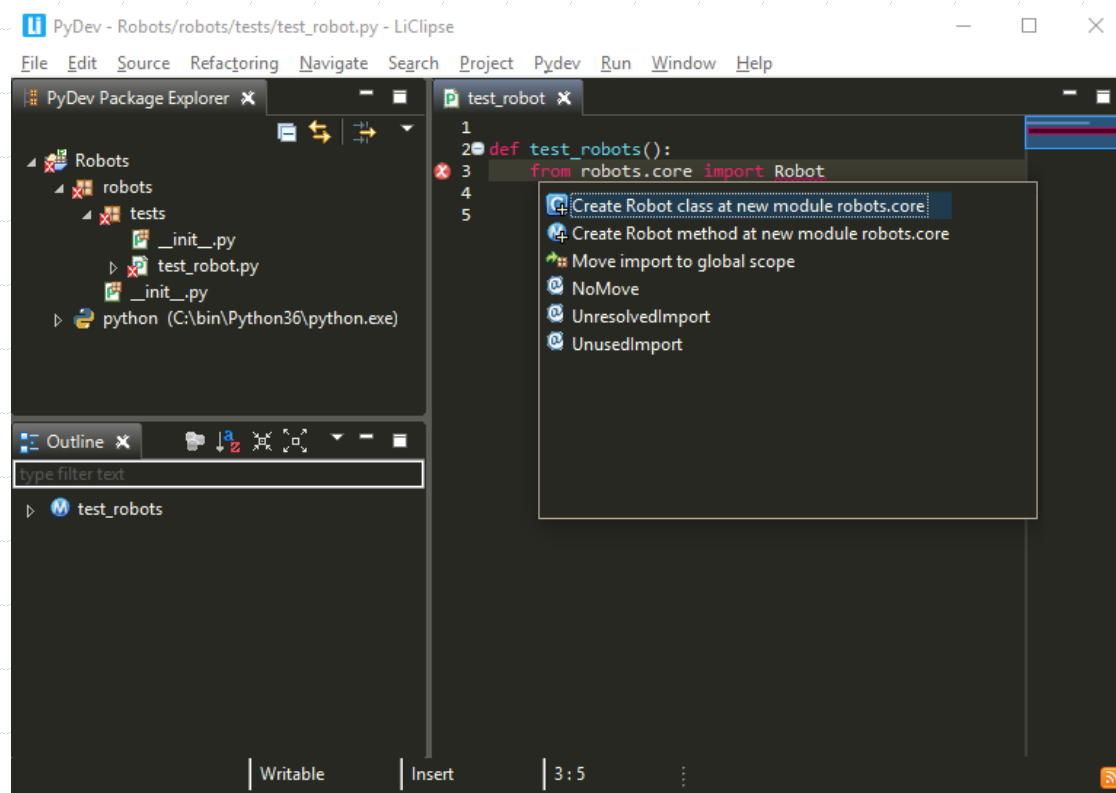
- File Explorer (Left):** Shows a project structure with folders like 'Plots' and 'IPythonConsole', and files such as 'plugin.py', 'chart_plot_example.py', and 'plugin.py - ipythonconsole'.
- Code Editor (Center):** Displays the source code for 'plugin.py - plots'. The code includes comments, imports, and a class definition for 'Plots(SpyderDockablePlugin)'.
- Variable Explorer (Right):** A table showing the current state of variables in the workspace.
- Plots Panel (Bottom Right):** Displays a 3D surface plot and a corresponding polar plot.

Name	Type	Size	Value
bool	bool	1	True
data	Array of str128	(3, 3)	ndarray object of numpy module
datetime_object	datetime	1	2021-04-14 17:35:14.687085
df	DataFrame	(2, 2)	Column names: Col1, Col2
filename	str	53	/Users/Documents/spyder/spyder/tests/test_dont_use.py
li	list	5	['abcd', 745, 2.23, 'efgh', 70.2]
myset	set	3	{'2', '1', '3'}
r	float	1	6.46567886443
t	tuple	5	('abcd', 745, 2.23, 'efgh', 70.2)
tinylist	list	2	[123, 'efgh']
x	float64	1	1.1235123099439

At the bottom of the interface, the status bar indicates: conda: spyder-dev, Python 3.8.5, LSP Python: ready, master, Line 10, Col 1, UTF-8, LF, RW, Mem 64%.

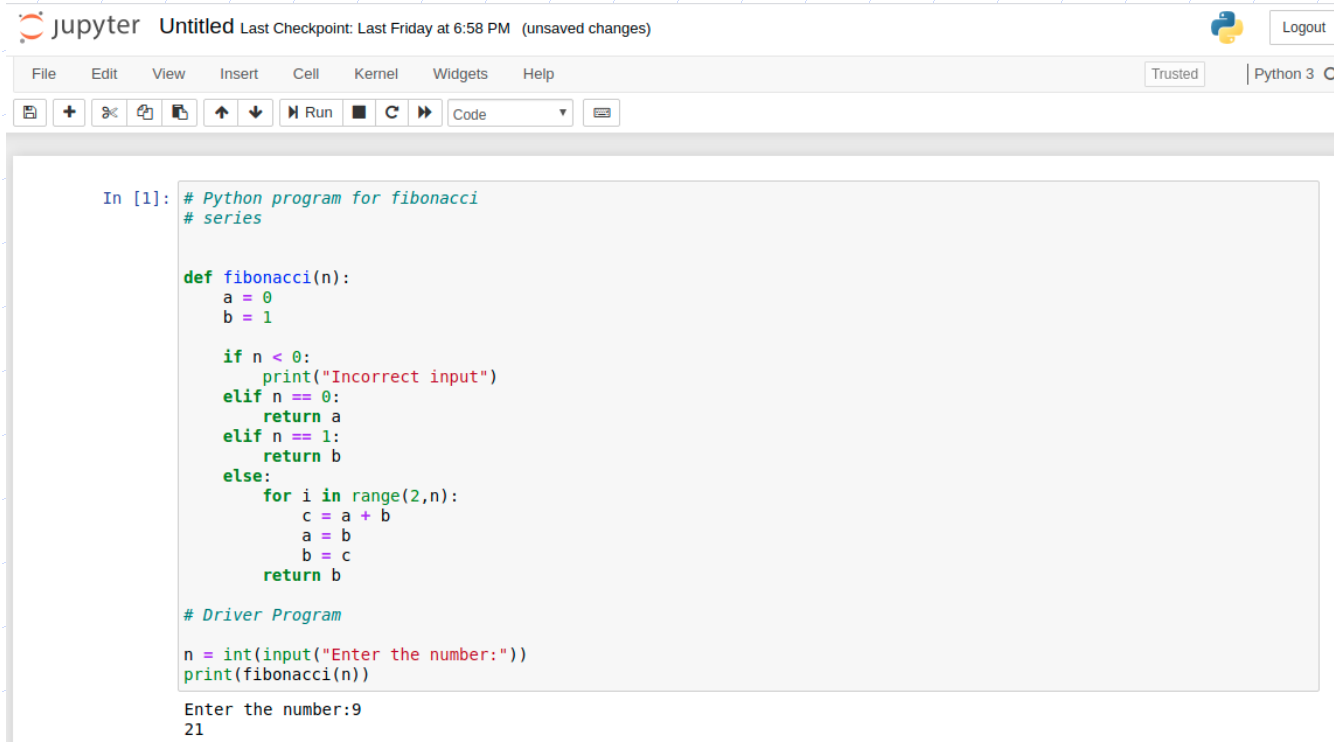
PyDev

PyDev je plugin za poznati Eclipse IDE, otvorenog je koda i namenjen programiranju za Python, Jython i IronPython, Eclipse pruža mnoge dodatke koji su izuzetno prilagodljivi i skalabilni a podržava mnoge programske jezike, uključujući Java, C, C++, Python, Ruby. Instalacija plugina u Eclipse je zahtevniji postupak, nije IDE okruženje za početnike, zvanična stranica za preuzimanje je <https://www.pydev.org/index.html>



Jupyter

Jupyter je u formi interaktivne radne sveske (notebook), fajlovi imaju ekstenziju .ipynb (*Interactive Python Notebook*) i izdeljeni su na ćelije (*cells*) koje dodajemo sa "+" u toolbaru, za ćelije namenjene za tekst biramo iz padajućeg menija Markdown, a za ćelije za programski kod biramo Code, u pozadini Jupytera (backend) se izvršava Python kernel na kome se izvršava sav kôd koji je napisan u Code ćelijama, zvanični sajt je <https://jupyter.org/>



The screenshot displays the Jupyter Notebook web interface. At the top, the title bar shows 'Jupyter' with its logo, followed by 'Untitled' and 'Last Checkpoint: Last Friday at 6:58 PM (unsaved changes)'. On the right, there is a 'Logout' button. Below the title bar is a menu bar with options: File, Edit, View, Insert, Cell, Kernel, Widgets, and Help. A 'Trusted' status indicator and 'Python 3' are also visible. The main toolbar contains icons for file operations (new, open, save, download, upload), cell navigation (up, down), and execution (run, interrupt, stop). The central area shows a code cell with the following Python code:

```
In [1]: # Python program for fibonacci
# series

def fibonacci(n):
    a = 0
    b = 1

    if n < 0:
        print("Incorrect input")
    elif n == 0:
        return a
    elif n == 1:
        return b
    else:
        for i in range(2,n):
            c = a + b
            a = b
            b = c
        return b

# Driver Program

n = int(input("Enter the number:"))
print(fibonacci(n))
```

Below the code, the input 'Enter the number:9' is shown, followed by the output '21'.

Python verzije jezika

- Dve važne verzije jezika Python su 2.x i 3.x
- Python verzija 2.7.18 je izašla u aprilu 2020. i predstavljala je kraj za razvoj Python 2 jezika
- Velika količina koda napisana u verziji 2.x
- Verzija 3.x donosi značajna poboljšanja na nivou jezika
 - Bolje definisani konstrukti, jednostavnija sintaksa za usvajanje
 - Nešto drugačiji način upotrebe pojedinih operatora
 - Izmene u ponašanju pojedinih tipova podataka
- U okviru ovog predmeta koristi se verzija 3.x
- Problemi kompatibilnosti koda (programi pisani u verziji 3.x se ne izvršavaju u 2.x verzijama, dok programi pisani u 2.x verzijama mogu imati problema tokom izvršavanja u 3.x verzijama)
- Postoji alat za translaciju koda iz jedne verzije u drugu

Python aktuelna verzija

- Na adresi <https://www.python.org/downloads/> može se preuzeti instalacioni fajl programskog jezika Python za odgovarajući operativni sistem
- Oktobar 2025, aktuelna Python verzija je 3.14.0



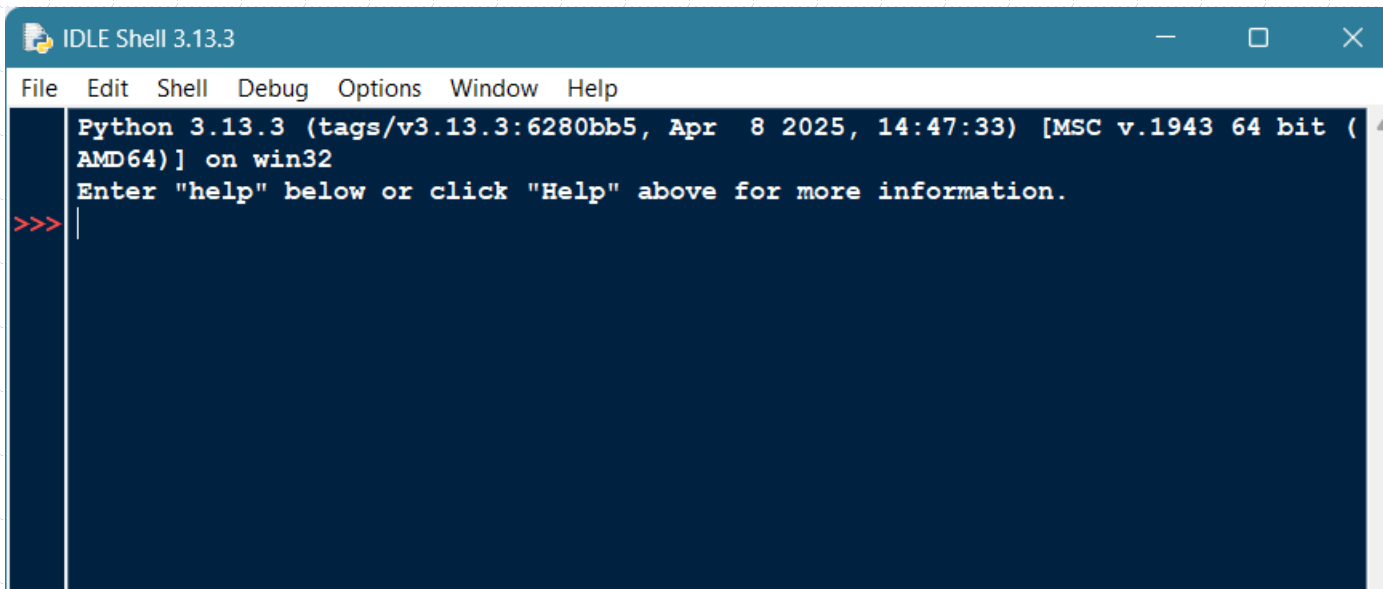
2. Numerički podaci i promenljive

Python osobine kôda

- Interpretirani programski jezik
- Razlikuje mala i velika slova u kôdu (*case sensitive*)
- Promenljive se eksplicitno ne deklarišu
- Ne postoji terminator naredbi
 - Uobičajeno se piše jedna naredba u jednom redu
 - Ukoliko se piše više naredbi u jednom redu, koristi se separator naredbi tačka zarez (;)
- Blokovi kôda se zadaju identacijom (uvlačenjem)
 - Koristi se za grupisanje naredbi u blokove
 - Sve naredbe uvučene isti broj belih znakova čine jedan blok kôda
- Dozvoljeno je pisanje praznih linija i one se ignorišu u kôdu

IDLE

- U osnovnoj Python instalaciji je razvojno okruženje IDLE, koje uključuje interaktivni interpreter, editor teksta i alate za testiranje programskog kôda
- Nakon pokretanja, otvara se sledeći prozor:



```
IDLE Shell 3.13.3
File Edit Shell Debug Options Window Help
Python 3.13.3 (tags/v3.13.3:6280bb5, Apr 8 2025, 14:47:33) [MSC v.1943 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>> |
```

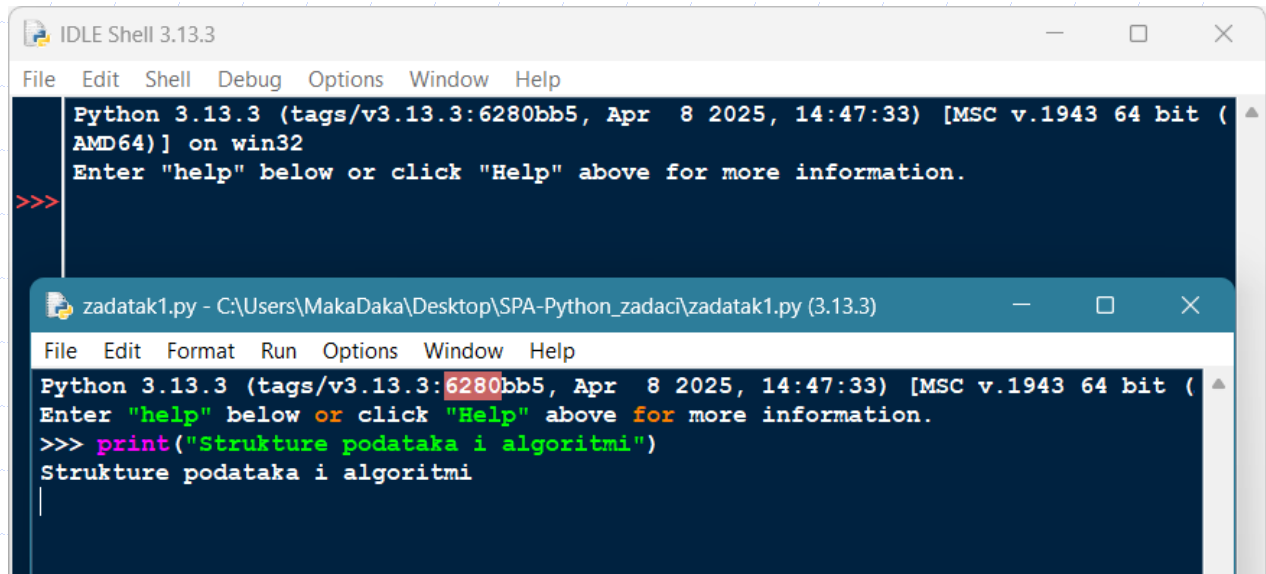


- Izlazak iz IDLE: ugrađene funkcije `quit()` ili `exit()` ili zatvaranjem aplikacije

IDLE

- IDLE nije pogodan za izvršavanje složenijeg koda i zato se naredbe grupišu u program koji je zapisan na spoljašnjoj memoriji (SSD, HDD)
- Programski kod se zapisuje u fajl-skript sa ekstenzijom **.py**
- Program se pokreće ili u IDLE-u ili se izvršava kao skripta

Po snimanju na disk, u datoteku sa ekstenzijom **.py** (u primeru **zadatak1.py**), program se pokreće izborom opcije **Run Module**, iz menija **Run** ili pritiskom na **F5**



The image shows two overlapping windows from the IDLE 3.13.3 environment. The top window, titled 'IDLE Shell 3.13.3', displays the Python interpreter's startup screen with the prompt 'Python 3.13.3 (tags/v3.13.3:6280bb5, Apr 8 2025, 14:47:33) [MSC v.1943 64 bit (AMD64)] on win32' and the instruction 'Enter "help" below or click "Help" above for more information.' Below this, the prompt '>>>' is visible. The bottom window, titled 'zadatak1.py - C:\Users\MakaDaka\Desktop\SPA-Python_zadaci\zadatak1.py (3.13.3)', shows the same Python interpreter interface but with a script being executed. The script contains the line '>>> print("Struktura podataka i algoritmi")', and the output 'Struktura podataka i algoritmi' is displayed below the prompt.

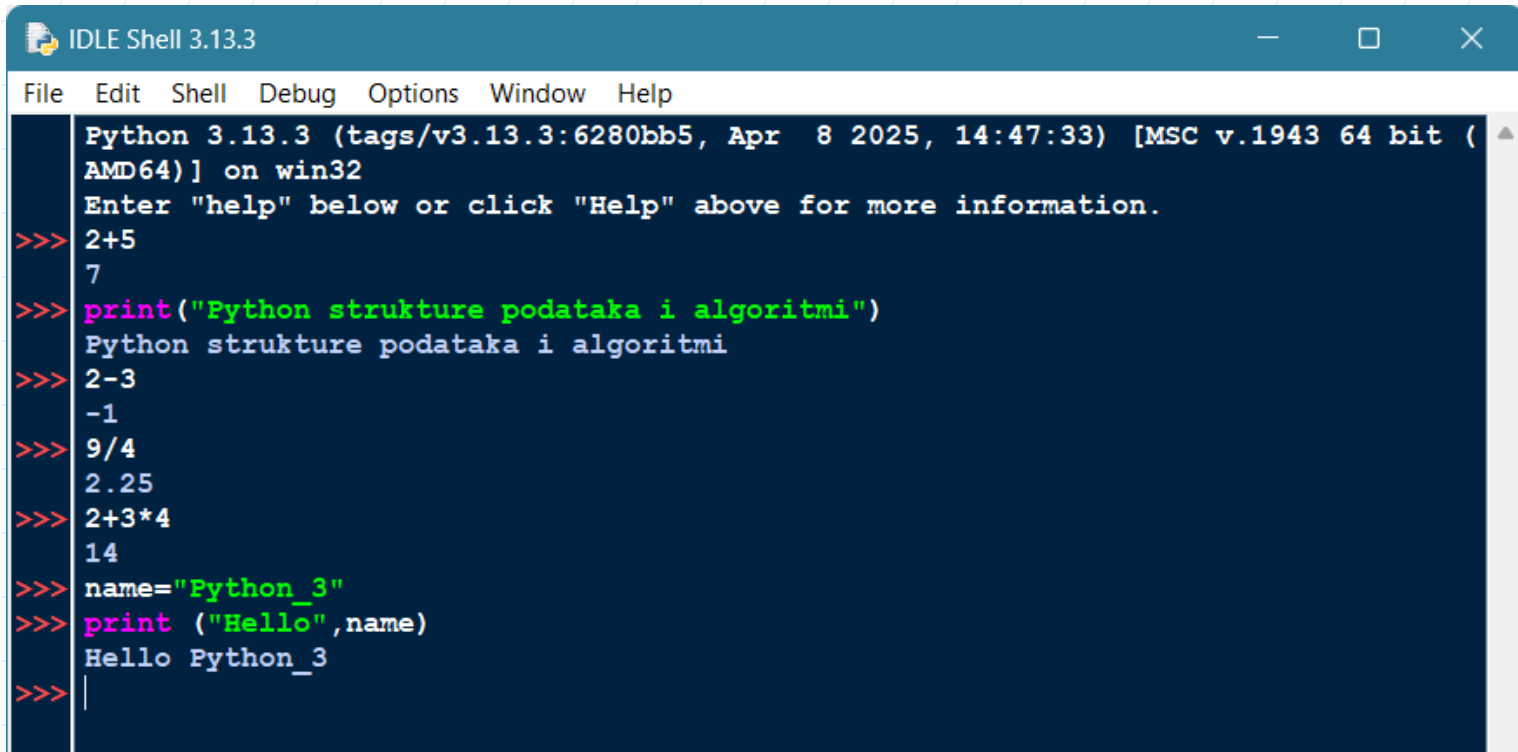
- Pokrenuti skriptu iz konzole (komandne linije)
 - Windows: **python zadatak1.py**
 - Linux: **python3 zadatak1.py**

```
#!/usr/bin/env python3  
print("HelloWorld!")
```

Oznaka **#!** u skripti predstavlja tzv. **shebang** i koristi se na Linux sistemima za lociranje interpretera

IDLE

- Interaktivno okruženje omogućava da se odmah testira deo koda koji se napiše
- Simbol `>>>` označava tekuću liniju za unos komande koja se po pritisku na taster `<Enter>` izvršava od strane interpretera, a njen rezultat ispisuje u sledećem redu
- `print()` je funkcija za ispis



```
IDLE Shell 3.13.3
File Edit Shell Debug Options Window Help
Python 3.13.3 (tags/v3.13.3:6280bb5, Apr 8 2025, 14:47:33) [MSC v.1943 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>> 2+5
7
>>> print("Python strukture podataka i algoritmi")
Python strukture podataka i algoritmi
>>> 2-3
-1
>>> 9/4
2.25
>>> 2+3*4
14
>>> name="Python_3"
>>> print ("Hello",name)
Hello Python_3
>>> |
```

2.1. Numerički podaci

- Python podržava 3 numerička tipa podataka: celobrojni, realni i kompleksni
- Celobrojni su označeni celi brojevi, realni su brojevi u pokretnom zarezu, a kompleksni su u obliku $x + yj$ gde x i y predstavljaju realne brojeve, a j označava imaginarni deo

tip	oznaka	domen vrednosti	primeri vrednosti
celobrojni	int	celi brojevi	0, 1, -1243
realni	float	realni brojevi	1.25, .02, -145.25
kompleksi	complex	kompleksi brojevi	-3j, -0.5 – 4.25j

- Drugi oblici numeričkih konstanti:
 - Int (binarne konstante prefiks 0b ili 0B, oktalne konstante prefiks 0o ili 0O, heksadecimalne konstante prefiks 0x ili 0X)
 - Float (sufiks e ili E za eksponent)

Aritmetičke operacije, operatori dodele i poređenja

Aritmetičke operacije

Znak	Operacija
+	Sabiranje
-	Oduzimanje
*	Množenje
/	Deljenje
%	Ostatak pri deljenju
//	Celobrojno deljenje
**	Stepenovanje

Operator	Ponašanje
+=	$c += a \leftrightarrow c = c + a$
-=	$c -= a \leftrightarrow c = c - a$
*=	$c *= a \leftrightarrow c = c * a$
/=	$c /= a \leftrightarrow c = c / a$
%=	$c \% = a \leftrightarrow c = c \% a$
**=	$c ** = a \leftrightarrow c = c ** a$
//=	$c //= a \leftrightarrow c = c // a$

Operatori poređenja

== jednakost
!= nejednakost
> veće
< manje
>= veće ili jednako
<= manje ili jednako

Operatori dodele

- Kombinuju operaciju sa dodelom vrednosti
- Operacija se sprovedi nad levim i desnim operandom
- Rezultat se smešta u levi operand

Prioritet operacija **PEMDAS**
(), ******, *****, **/**, **+**, **-**

Aritmetički izrazi - primeri

```
>>>1.0+2    #Sabiranje float i int daje float
3.0
>>>2*8.0    #Množenje float i int daje float
16.0
>>>int(5.0/2)  #Int odbacuje decimalni deo
2
>>>20%7      #Moduo (ostatak deljenja)
6
>>>5 % -3    #r=x-(y*(x//y))  5/-3=-2, -2*-3=6, 5-6=-1
-1
>>>100/2*2   #Na istom nivou prioriteta * i /, s leva u desno
100.0
>>>0.1+0.2
0.30000000000000004
>>>round(2.71828,2)
2.72
```

Numerički podaci

```
IDLE Shell 3.13.3
File Edit Shell Debug Options Window Help
Python 3.13.3 (tags/v3.13.3:6280bb5, Apr 8 2025, 14:47:33) [MSC v.1943 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>> type(1) #tip podatka
<class 'int'>
>>> num = 25
>>> int("25") #Konverzija stringa u broj
25
>>> 1000000
1000000
>>> 1_000_000 #Donja crta za razdvajanje klasa broja
1000000
>>> type(1.0)
<class 'float'>
>>> 1_000_000.0
1000000.0
>>> 1e6 #Exponencijalni zapis broja
1000000.0
>>> 1e-4
0.0001
>>> 2e400 #Dostizanje gornje granice za broj
inf
>>> n = 1 + 2j #Kompleksni broj
>>> n
(1+2j)
>>> n.conjugate() #Konjug-kompleksni broj
(1-2j)
>>> |
```

Aritmetički izrazi, numerički podaci

- Logički operatori:

```
print(3 > 2 and 4 > 3) # True - because both statements are true
print(3 > 2 and 4 < 3) # False - because the second statement is false
print(3 < 2 and 4 < 3) # False - because both statements are false
print('True and True: ', True and True)
print(3 > 2 or 4 > 3) # True - because both statements are true
print(3 > 2 or 4 < 3) # True - because one of the statements is true
print(3 < 2 or 4 < 3) # False - because both statements are false
```

- Operatori poređenja:

```
print(3 > 2) # True, because 3 is greater than 2
print(3 >= 2) # True, because 3 is greater than 2
print(3 < 2) # False, because 3 is greater than 2
print(2 < 3) # True, because 2 is less than 3
print(2 <= 3) # True, because 2 is less than 3
print(3 == 2) # False, because 3 is not equal to 2
print(3 != 2) # True, because 3 is not equal to 2
print(len('mango') == len('avocado')) # False
print(len('mango') != len('avocado')) # True
print(len('mango') < len('avocado')) # True
print(len('milk') != len('meat')) # False
print(len('milk') == len('meat')) # True
print(len('tomato') == len('potato')) # True
print(len('python') > len('dragon')) # False
```

Konvertovanje tipa podatka

- Konvertovanje jednog tipa podatka u drugi (**casting**):

```
# int to float
num_int = 10
print('num_int', num_int)          # 10
num_float = float(num_int)
print('num_float:', num_float)    # 10.0

# float to int
gravity = 9.81
print(int(gravity))               # 9

# int to str
num_int = 10
print(num_int)                   # 10
num_str = str(num_int)
print(num_str)                   # '10'

# str to int or float
num_str = '10.6'
num_float = float(num_str)
print('num_float', float(num_str)) # 10.6
num_int = int(num_float)
print('num_int', int(num_int))     # 10

# str to list
first_name = 'Asabeneh'
print(first_name)                 # 'Asabeneh'
first_name_to_list = list(first_name)
print(first_name_to_list)         # ['A', 's', 'a', 'b', 'e', 'n', 'e', 'h']
```

Ugrađene funkcije

- Veliki broj ugrađenih matematičkih funkcija

Funkcija	Vraća
<code>abs(x)</code>	Apsolutno x
<code>math.ceil(x)</code>	Najbliži veći integer od broja x
<code>cmp(x, y)</code>	-1 if $x < y$, 0 if $x == y$, or 1 if $x > y$, u Pythonu 3 napisati <code>def cmp(a,b): return (a > b) - (a < b)</code>
<code>math.exp(x)</code>	e^x
<code>math.fabs(x)</code>	Apsolutno x
<code>math.floor(x)</code>	Nabliži manji integer od broja x
<code>math.log(x)</code>	$\ln x$, za $x > 0$
<code>math.log10(x)</code>	$\log x$, za $x > 0$.
<code>max(x1, x2, ...)</code>	Najveći argument
<code>min(x1, x2, ...)</code>	Namjanji argument
<code>math.modf(x)</code>	Kreira uređeni par realnih brojeva (frakcije, celobrojnog deo), <i>oba dela imaju isti predznak kao x.</i>
<code>pow(x, y)</code> , <code>math.pow(x, y)</code>	$x^{**}y$ gde je rezultat u drugom slučaju obavezno float
<code>round(x [,n])</code>	x zaokružen na n decimalnih cifara iza decim. tačke.
<code>math.sqrt(x)</code>	The square root of x for $x > 0$

Ugrađene funkcije

- Ugrađene trigonometrijske funkcije

Funkcija	Opis
<code>math.acos(x)</code>	Arkus kosinus od x u radijanima.
<code>math.asin(x)</code>	Arkus sinus od x u radijanima.
<code>math.atan(x)</code>	Arkus tangens od x u radijanima.
<code>math.atan2(y, x)</code>	Arkus tangens kao f(kateta_y,kateta_x) u radijanima.
<code>math.cos(x)</code>	Kosinus od x radijana.
<code>math.hypot(x, y)</code>	Euklidska norma, $\sqrt{x^2 + y^2}$, hipotenuza.
<code>math.sin(x)</code>	Sinus od x radijana.
<code>math.tan(x)</code>	Tangens od x radijana.
<code>math.degrees(x)</code>	Konverzija iz radijana u stepene.
<code>math.radians(x)</code>	Konverzija iz stepena u radijane.

Ugrađene funkcije - primeri

```
IDLE Shell 3.13.3
File Edit Shell Debug Options Window Help
Python 3.13.3 (tags/v3.13.3:6280bb5, Apr 8 2025, 14:47:33) [MSC v.1943 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>> abs(-45)
45
>>> abs(100.12)
100.12
>>> import math
>>> math.ceil(-45.17) #Najblizi veci integer od broja
-45
>>> math.ceil(math.pi)
4
>>> math.exp(-45.17) #Funkcije e^x
2.4150062132629406e-20
>>> math.floor(-45.17) #Najblizi manji integer od broja
-46
>>> math.log(119)
4.77912349311153
>>> max(-80,-20,-10)
-10
>>> min(-80,-20,-10)
-80
>>> math.pow(100,2)
10000.0
>>> round(80.23456,2)
80.23
>>> acos(-1)
Traceback (most recent call last):
  File "<pyshell#12>", line 1, in <module>
    acos(-1)
NameError: name 'acos' is not defined
>>> math.acos(-1)
3.141592653589793
>>> |
```

2.2. Promenljive

- Promenljiva (*variable*) je podatak kojem je moguće menjati vrednost
- Promenljiva zauzima prostor u memoriji, i sadržaj memorije se tumači prema tipu primenljive
- Memorijskoj lokaciji se pristupa preko imena promenljive (identifikator)
- Promenljiva se ne definiše eksplicitno već dinamički nakon prve upotrebe, a ostatak programa je zatim može koristiti
- Može se koristiti specijalna vrednost **None** da označi odsustvo vrednosti
- Promenljivoj se može redefinisati tip
- Promenljiva može biti lokalna ili globalna:
 - Definiše opseg vidljivosti
 - Bitno kod upotrebe potprograma

Imenovanje promenljivih

- Ime promenljive može biti proizvoljne dužine
- Ime promenljive sme da sadrži samo:
 - Brojeve
 - Velika i mala slova
 - Donju crticu „_“ (*underscore*)
- Ime ne sme biti rezervisana reč
- Ime ne sme početi brojem
- Ne može da sadrži specijalne znake tipa !, @, #, \$
- Razlika za mala i velika slova

- Primeri dozvoljenih imena:

jabuka, slova123, A4, ime_psa



- Primeri nedozvoljenih imena:

ime psa, 123slova, jabuka#



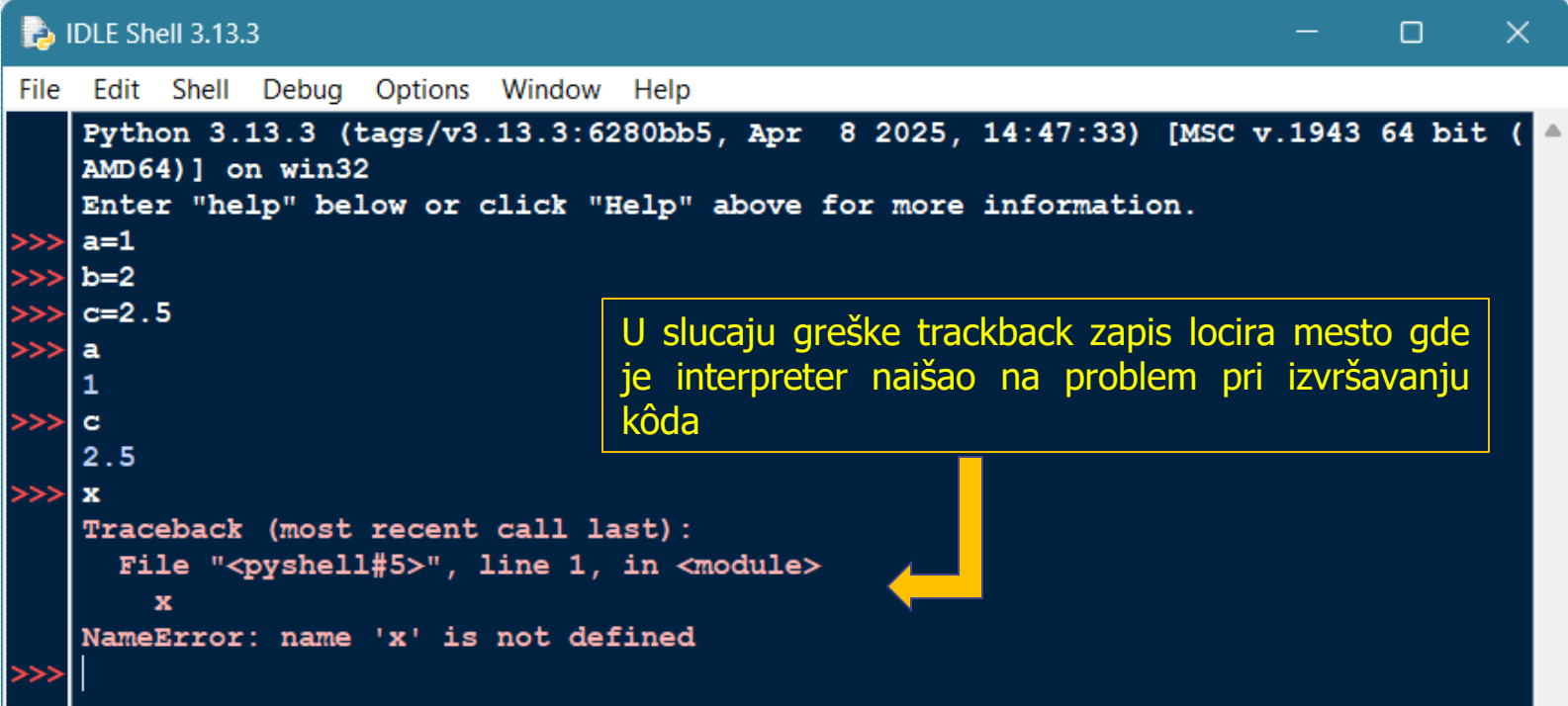
Python rezervisane reči

Python 3 ima 33 rezervisane reči koje ne mogu biti upotrebljene kao identifikatori promenljivih

False	class	finally	is	return
None	continue	for	lambda	try
True	def	from	nonlocal	while
and	del	global	not	with
as	elif	if	or	yield
assert	else	import	pass	
break	except	in	raise	

Dodela vrednosti promenljivim

```
>>> a = 1
>>> b = 2
>>> c = 2.5
>>> x      #Nije dodeljena vrednost za x!!!! (GREŠKA)
```



U slučaju greške traceback zapis locira mesto gde je interpreter naišao na problem pri izvršavanju kôda

```
IDLE Shell 3.13.3
File Edit Shell Debug Options Window Help
Python 3.13.3 (tags/v3.13.3:6280bb5, Apr 8 2025, 14:47:33) [MSC v.1943 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>> a=1
>>> b=2
>>> c=2.5
>>> a
1
>>> c
2.5
>>> x
Traceback (most recent call last):
  File "<pyshell#5>", line 1, in <module>
    x
NameError: name 'x' is not defined
>>> |
```

Dodela vrednosti promenljivim

- Deklarisanje i definisanje promenljive se vrši prilikom prve dodele vrednosti promenljivoj preko operatora „=“
- Moguća je višestruka dodela vrednosti
- Ugrađena metoda `type()` vraća tip promenljive

```
IDLE Shell 3.13.3
File Edit Shell Debug Options Window Help
Python 3.13.3 (tags/v3.13.3:6280bb5, Apr  8 2025, 14:47:33) [MSC v.1943 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>> osnova=2
>>> pi=3.14
>>> skola="Akademija_Sabac"
>>> print(osnova,pi,skola)
2 3.14 Akademija_Sabac
>>> a=b=c=1
>>> print(osnova, pi, skola,a,b,c)
2 3.14 Akademija_Sabac 1 1 1
>>> type(pi)
<class 'float'>
>>> type(a,b,c)
Traceback (most recent call last):
  File "<pyshell#7>", line 1, in <module>
    type(a,b,c)
TypeError: type.__new__() argument 1 must be str, not int
>>> type(c)
<class 'int'>
>>> type(skola)
<class 'str'>
>>> |
```

Uništavanje promenljivih

- Uglavnom se izvršava dinamički
- Python poseduje sakupljač đubreta (*garabage collector*) koji vodi računa o nepotrebnim podacma
- Eksplicitnom naredbom `del` može se uništiti promenljiva

```
>>> a = 100
>>> print(a)
```

```
100
```

```
>>> del(a)
>>> print(a)
```

```
Traceback (most recent call last):
```

```
  File "<pyshell#21>", line 1, in <module>
    print(a)
```

```
NameError: name 'a' is not defined
```

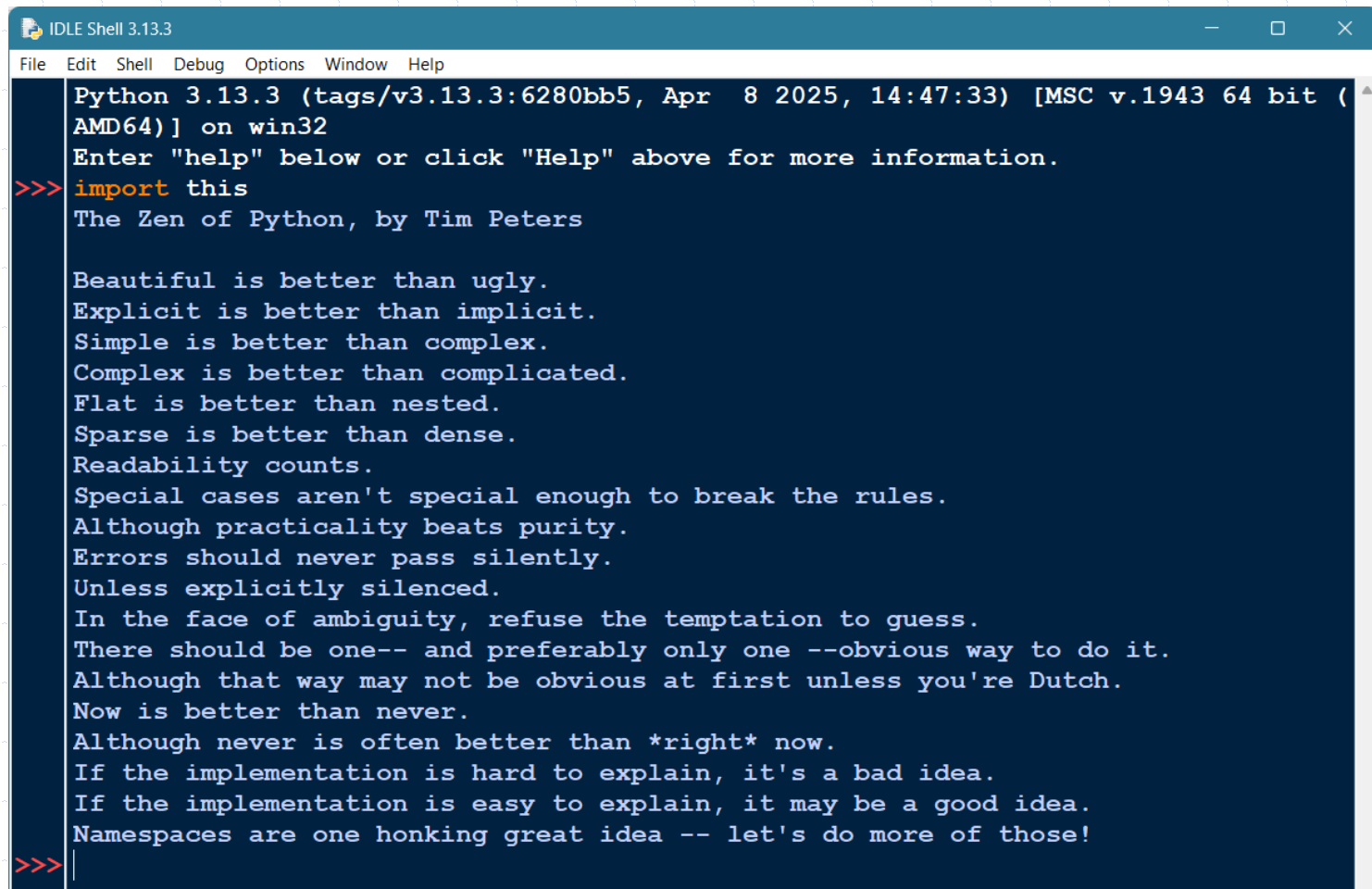
Interpreter odmah prekida izvršavanje programa!

Komentari u kodu

- Dva načina za zadavanje komentara u kodu:
- Jednoredni komentari iza simbola # (*hash*)
`#primer jednorednog komentara`
- Python ne analizira sadržaj od znaka # do kraja reda
- Višeredni komentari unutar trostrukih navodnika `"""` ili `'''` koji predstavljaju *heredocs/docstrings* navodnike koji se najčešće upotrebljavaju za dokumentovanje funkcija
`""" Ovo je
višeredni komentar.
Zadržava sve bele znakove i
prelaske u novi red """`

The Zen of Python

Filozofija Python jezika sadržana je u skupu principa Tima Petersa



```
IDLE Shell 3.13.3
File Edit Shell Debug Options Window Help
Python 3.13.3 (tags/v3.13.3:6280bb5, Apr  8 2025, 14:47:33) [MSC v.1943 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>> import this
The Zen of Python, by Tim Peters

Beautiful is better than ugly.
Explicit is better than implicit.
Simple is better than complex.
Complex is better than complicated.
Flat is better than nested.
Sparse is better than dense.
Readability counts.
Special cases aren't special enough to break the rules.
Although practicality beats purity.
Errors should never pass silently.
Unless explicitly silenced.
In the face of ambiguity, refuse the temptation to guess.
There should be one-- and preferably only one --obvious way to do it.
Although that way may not be obvious at first unless you're Dutch.
Now is better than never.
Although never is often better than *right* now.
If the implementation is hard to explain, it's a bad idea.
If the implementation is easy to explain, it may be a good idea.
Namespaces are one honking great idea -- let's do more of those!
>>> |
```

3. Python strukture podataka

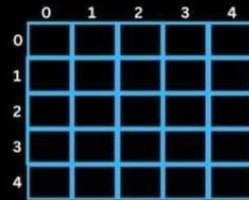
Array



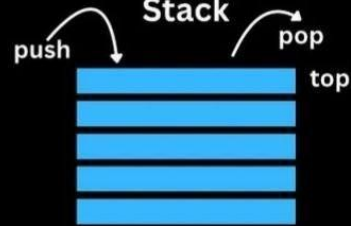
Queue



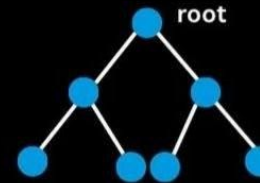
Matrix



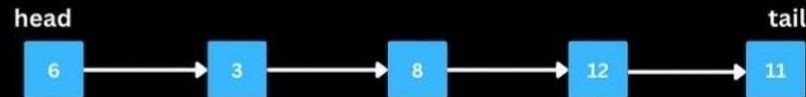
Stack



Tree

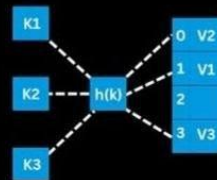


Linked List

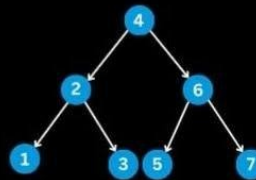


@visualcoders

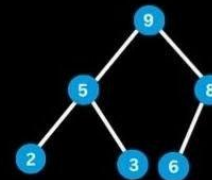
HashMap



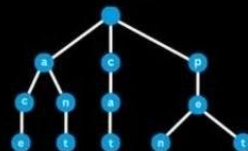
BST



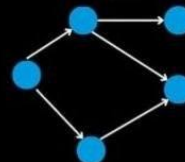
Heap



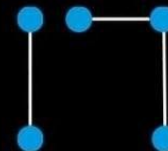
Trie



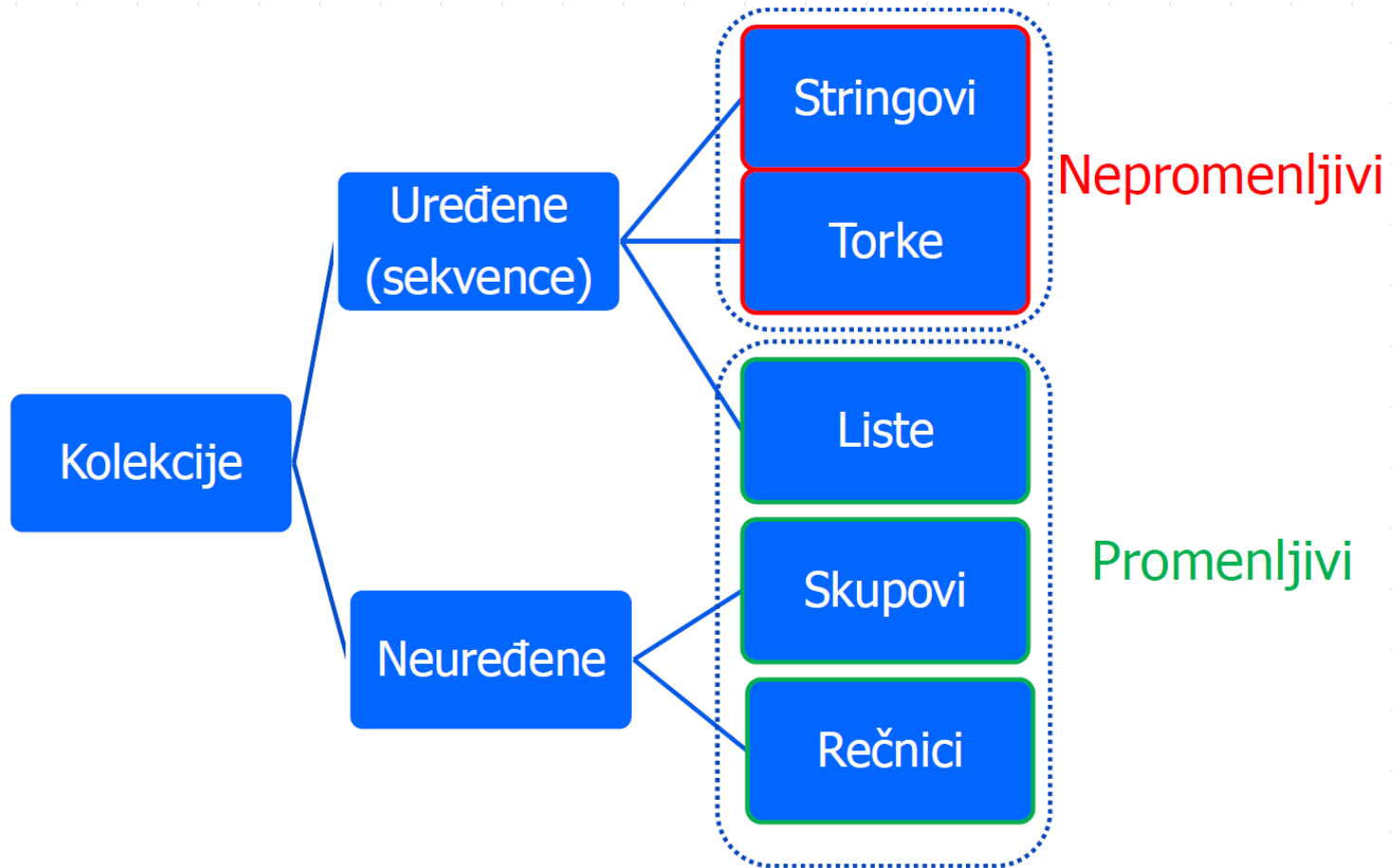
Graph



Union Find



Python kolekcije podataka



3.1 String

- Tekstualne sekvence predstavljaju tekstualni objekat tipa **String**
- String je niz karaktera ograničen jednostrukim ili dvostrukim navodnicima (apostrofima)
- String u više redova se navodi trostrukim navodnicima ('''')
- String se kreira dodelom vrednosti ili konstruktorom `str()` npr.
`>>>str(3.14)` daje `'3.14'`
`>>>str(True)` daje `'True'`
- String je sačinjen od znakova (karaktera) koji imaju svoju numeričku (binarnu) formu u memoriji računara
- ASCII sistem kodira znakove sa 7 bita i sadrži 128 znakova, dok prošireni ASCII kodira sa 8 bita, odnosno sadrži 256 znakova
- Python koristi 16-bitni Unicode standard, dok se za samo kodiraje koristi se UTF8 sistem, gde se svaki karakter predstavlja zapisom od 8 do 32 bita

String

- Python prepoznaje jedinstveni UCP kodni broj u Unicode sistemu
- UCP se dobija pomoću funkcije `ord(znak)`
- Znak se dobija pomoću `chr(UCP_broj)`
- Dužina stringa dobija se pomoću `len(string)`

```
>>> ord('A')
65
>>> chr(1096)
'𐀖'
>>> ord('𐀖')
1096
>>> len("Alfanumerik")
11
>>>
```

- Kreiranje stringa:

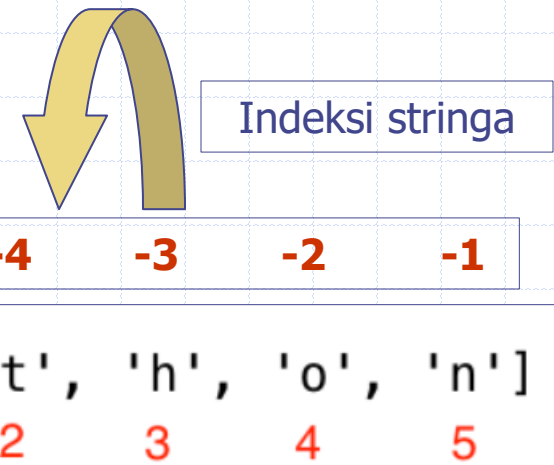
```
>>> var1='Hello World'           #Jednostruki navodnici
>>> var2="Python Programming"    #Dvostruki navodnici
```

- Pristup karakteru stringa:

```
>>> print("var1[0]:", var1[0])
>>> print("var2[1:5]:", var2[1:5])
```

var1[0]: H

var2[1:5]: ytho



String

- Kreiranje stringa:

```
letter = 'P'  
print(letter)           # P  
print(len(letter))      # 1  
greeting = 'Hello, World!'  
print(greeting)         # Hello, World!  
print(len(greeting))    # 13
```

- Spajanje (konkatenacija) stringova:

```
first_name = 'Asabeneh'  
last_name = 'Yetayeh'  
space = ' '  
full_name = first_name + space + last_name  
print(full_name) # Asabeneh Yetayeh  
# Checking the length of a string using len() built-in function  
print(len(first_name)) # 8  
print(len(last_name))  # 7  
print(len(first_name) > len(last_name)) # True  
print(len(full_name))  # 16
```

String

- Ažuriranje stringa:

```
>>>var1='Hello World'
```

```
>>>print("Azuriran string:", var1[:6] + Python)
```

Azuriran string: Hello Python

- Trostruki navodnici omogućuju proširenje stringa na više redova:

```
>>>para_str="""this is a long string that is made up of  
several lines and non-printable characters such as  
TAB (\t) and they will show up that way when displayed.  
NEWLINES within the string, whether explicitly given like  
this within the brackets [\n], or just a NEWLINE within  
the variable assignment will also show up.  
"""
```

```
>>>print(para_str)
```

```
this is a long string that is made up of  
several lines and non-printable characters such as  
TAB ( ) and they will show up that way when displayed.  
NEWLINES within the string, whether explicitly given like  
this within the brackets [  
, or just a NEWLINE within  
the variable assignment will also show up.
```

String

- Escape karakteri (sekvence):

\a (0x07) Bell	\b (0x08) Backspace	\cx \C-x	Control-x
\e (0x1b) Escape	\f (0x0c) Formfeed	\M-\C-x	Meta-Control-x
\n (0x0a) Newline	\nnn Octal notation	\r (0x0d)	Carriage return
\s (0x20) Space	\t (0x09) Tab	\v (0x0b)	Vertical tab
\x Character x	\xnn Hexadecimal notation		

Operator	Opis	Primer
+	Konkatenacija	a + b HelloPython
*	Ponavljanje	a*2 HelloHello
[]	Slice - uzima karakter datog indeksa	a[1] daje e
[:]	Range Slice - uzima karaktere datog opsega	a[1:4] daje ell
in	Membership - true ako karakter pripada stringu	H in a True
not in	Membership - suprotno od prethodnog	M not in a True
r/R	Raw String - ukida značenje escape karaktera. Ovaj operator (r ili R) ide pre prvog navoda stringa.	print (r'\n') \n print (R'\n') \n

```
>>>a='Hello'
>>>b='Python'
```

String

- Raspakivanje karaktera stringa u niz promenljivih:

```
language = 'Python'
a,b,c,d,e,f = language # unpacking sequence characters into variables
print(a) # P
print(b) # y
print(c) # t
print(d) # h
print(e) # o
print(f) # n
```

- Pristup karakterima stringova:

```
language = 'Python'
first_letter = language[0]
print(first_letter) # P
second_letter = language[1]
print(second_letter) # y
last_index = len(language) - 1
last_letter = language[last_index]
print(last_letter) # n
```



```
language = 'Python'
last_letter = language[-1]
print(last_letter) # n
second_last = language[-2]
print(second_last) # o
```

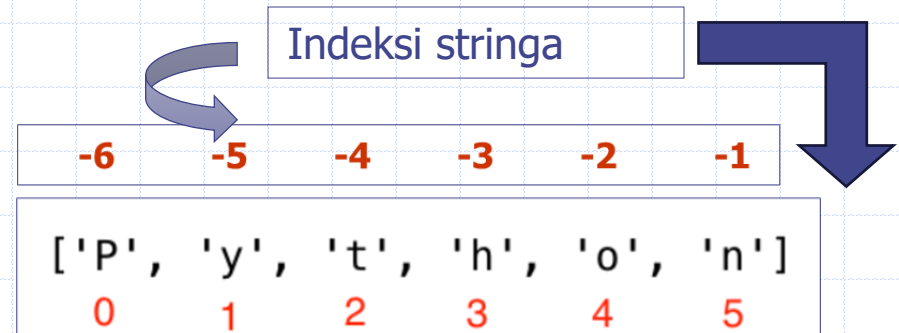
String

- String Slicing (uzimanje karaktera datog indeksa) :

```
greeting = 'Hello, World!'
print(greeting[::-1]) # !dlroW ,olleH
```

```
language = 'Python'
pto = language[0:6:2] #
print(pto) # Pto
```

```
language = 'Python'
first_three = language[0:3] # starts at zero index and up to 3 but not include 3
print(first_three) #Pyt
last_three = language[3:6]
print(last_three) # hon
# Another way
last_three = language[-3:]
print(last_three) # hon
last_three = language[3:]
print(last_three) # hon
```



String metode

isdigit()

```
str = "123456"; # Only digit in this string
print (str.isdigit()); # True
str = "this is string !!!"; print (str.isdigit()); #False
```

islower()

```
str = "THIS is string example....wow!!!";
print (str.islower()); # False
str = "this is string example....wow!!!";
print (str.islower()); # True
```

isnumeric()

```
str = u"this2009";
print (str.isnumeric()); # False
str = u"23443434";
print (str.isnumeric()); # True
```

isspace()

```
str = "   ";
print (str.isspace()); # True
str = "This is string example....wow!!!";
print (str.isspace()); # False
```

String metode

istitle()

```
str = "This Is String Example...Wow!!!";  
print (str.istitle()); # True  
str = "This is string example....wow!!!";  
print (str.istitle()); # False
```

isupper()

```
str = "THIS IS STRING EXAMPLE....WOW!!!";  
print (str.isupper()); # True  
str = "THIS is string example....wow!!!";  
print (str.isupper()); # False
```

join(seq)

```
str = "--"; # spona  
seq = ("a", "b", "c"); # sekvenca stringova  
print (str.join( seq )); # a--b--c
```

len(string)

```
str = "this is string example....wow!!!";  
print ("Length of the string: ", len(str));
```

Length of the string: 32

String metode

max(str)

```
str = "this is really a string example....wow!!!";  
print ("Max character: " + max(str)); # Max character: y  
str = "this is a string example....wow!!!";  
print ("Max character: " + max(str)); # Max character: x
```

min(str)

```
str = "this-is-real-string-example....wow!!!";  
print ("Min character: " + min(str)); # Min character: !
```

replace(oldsub, newsub [, maxrepl])

```
str = "this is string example....wow!!! this is really string";  
print (str.replace("is", "was"));  
print (str.replace("is", "was", 3));  
thwas was string example....wow!!! thwas was really string  
thwas was string example....wow!!! thwas is really string
```

rstrip()

```
str = "    this is string example....wow!!! ";  
print (str.rstrip());  
str = "88888888this is string example....wow!!!88888888";  
print (str.rstrip('8'));  
    this is string example....wow!!!  
    this is string example....wow!!!88888888
```

3.2. Liste

- Jedna od osnovnih struktura Python podataka je **LISTA**
- To je indeksirana kolekcija povezanih objekata ili elemenata
- **Elementi liste mogu biti različitog tipa**
- Liste su u Pythonu dinamičke jer se mogu menjati i ne heterogene jer ne mora unapred da se deklarise tip elemenata u okviru nje
- Liste su promenljive (*mutable*) jer mogu u svakom trenutku da se menjaju dodavanjem, uklanjanjem ili menjanjem elemenata
- **Elementi liste su razdvojeni zarezom, u uglastim zagradama []**
- Pristup elementu liste vrši se navođenjem njegovog indeksa
- **Pozicija indeksa počinje od 0, tj. prvi element liste ima indeks 0**

```
['banana', 'orange', 'mango', 'lemon']  
  0         1         2         3
```

Liste

- Kreiranje liste:

```
>>>empty_lst=list() #prazna lista, preko ugrađ. funkcije list
>>>empty_lst=[]
>>>print(len(empty_list)) # 0
>>>zivotinje=['pas','macka','mis','majmun','slon']
>>>print (zivotinje) # ['pas', 'macka', 'mis', 'majmun', 'slon']
>>>list1,list2=[123,'xyz'],[456,'abc'] #kreiranje 2 liste
>>>list3=list2+[789]
```

- Primeri liste:

```
[1, 4, 17, 256, 3, 59, 45] #lista celih brojeva
['april','jun','septembar','novembar'] #lista stringova
[] #prazna lista
['Marko', 28, 101, 'Python'] #lista elemenata različitog tipa
[1, 4, [7, 6, [13]], [9, 5]] #lista čiji elementi su liste
```

Liste

- Kreiranje liste:

```
fruits = ['banana', 'orange', 'mango', 'lemon']  
vegetables = ['Tomato', 'Potato', 'Cabbage', 'Onion', 'Carrot']
```

```
# Print the lists and its length  
print('Fruits:', fruits)  
print('Number of fruits:', len(fruits))  
print('Vegetables:', vegetables)  
print('Number of vegetables:', len(vegetables))
```

metoda **len()**
vraća broj
elemenata liste

output

```
Fruits: ['banana', 'orange', 'mango', 'lemon']  
Number of fruits: 4  
Vegetables: ['Tomato', 'Potato', 'Cabbage', 'Onion', 'Carrot']  
Number of vegetables: 5
```

Liste

- Kreiranje liste putem opsega elemenata:

```
>>>lista=list(range(1,10)) #Kreiranje liste metodom range
```

```
>>>print (lista)
```

```
[1, 2, 3, 4, 5, 6, 7, 8, 9]
```

```
>>>lista=list(range(1,10,2)) #Lista 1-9, svaki drugi element
```

```
>>>print (lista)
```

```
[1, 3, 5, 7, 9]
```

```
>>>lista=[10,9,8,7,-1,-2,-3]
```

```
>>>print (lista[2], lista[-1])
```

```
>>>print (lista[1:5])
```

```
>>>print (lista[1:5:2])
```

```
>>>print (lista[12])
```

```
8 -3
```

```
[9,8,7,-1]
```

```
[9,7]
```

```
IndexError: list index out of range
```

Negativni indeks

-4 -3 -2 -1

['banana', 'orange', 'mango', 'lemon']

0 1 2 3

Pozitivni indeks

Liste

- Pristup elementima liste:

```
fruits = ['banana', 'orange', 'mango', 'lemon']
first_fruit = fruits[-4]
last_fruit = fruits[-1]
second_last = fruits[-2]
print(first_fruit)      # banana
print(last_fruit)       # lemon
print(second_last)      # mango
```

- Raspakivanje elemenata liste:

```
lst = ['item1','item2','item3', 'item4', 'item5']
first_item, second_item, third_item, *rest = lst
print(first_item)       # item1
print(second_item)      # item2
print(third_item)       # item3
print(rest)              # ['item4', 'item5']
```

```
fruits = ['banana', 'orange', 'mango', 'lemon']
all_fruits = fruits[0:4] # it returns all the fruits
```

```
fruits = ['banana', 'orange', 'mango', 'lemon']
all_fruits = fruits[-4:] # it returns all the fruits
```

Liste

- Izmena liste (mutable) :

```
>>>lista=[2, 4, 6, 8]
>>>lista[1]=0
>>>lista[-1]=12
>>>print (lista)
[2, 0, 6, 12]
```



```
fruits = ['banana', 'orange', 'mango', 'lemon']
fruits[0] = 'avocado'
print(fruits)          # ['avocado', 'orange', 'mango', 'lemon']
fruits[1] = 'apple'
print(fruits)          # ['avocado', 'apple', 'mango', 'lemon']
last_index = len(fruits) - 1
fruits[last_index] = 'lime'
print(fruits)          # ['avocado', 'apple', 'mango', 'lime']
```

- Provera postojanja elementa u listi (in):

```
>>>fruits=['banana','orange','mango','lemon']
>>>does_exist='banana' in fruits
>>>print (does_exist) #True
```

- Dodavanje elementa na kraj liste (append):

```
fruits = ['banana', 'orange', 'mango', 'lemon']
fruits.append('apple')
print(fruits)          # ['banana', 'orange', 'mango', 'lemon', 'apple']
fruits.append('lime')  # ['banana', 'orange', 'mango', 'lemon', 'apple', 'lime']
print(fruits)
```

Liste

- Dodavanje elementa u listu (**insert**) :

```
# syntax
lst = ['item1', 'item2']
lst.insert(index, item)
```

```
fruits = ['banana', 'orange', 'mango', 'lemon']
fruits.insert(2, 'apple') # insert apple between orange and mango
print(fruits)             # ['banana', 'orange', 'apple', 'mango', 'lemon']
fruits.insert(3, 'lime')  # ['banana', 'orange', 'apple', 'lime', 'mango', 'lemon']
print(fruits)
```

- Spajanje listi (**+**, **extend**):

```
>>>A=[1, 2, 3, 4, 5]
>>>B=[6, 7, 8, 9]
>>>C=A+B
>>>print(C)
[1, 2, 3, 4, 5, 6, 7, 8, 9]
>>>print(A+B[1:-1])
[1, 2, 3, 4, 5, 7, 8, 9]
>>>B='Jovana'
>>>C=A+B
>>>print(C)
```

```
num1 = [0, 1, 2, 3]
num2= [4, 5, 6]
num1.extend(num2)
print('Numbers:', num1) # Numbers: [0, 1, 2, 3, 4, 5, 6]
```

TypeError: can only concatenate list (not "str") to list

Liste

- Sortiranje liste (**sort**, **sorted**) :

```
# syntax
lst = ['item1', 'item2']
lst.sort()           # ascending
lst.sort(reverse=True) # descending
```

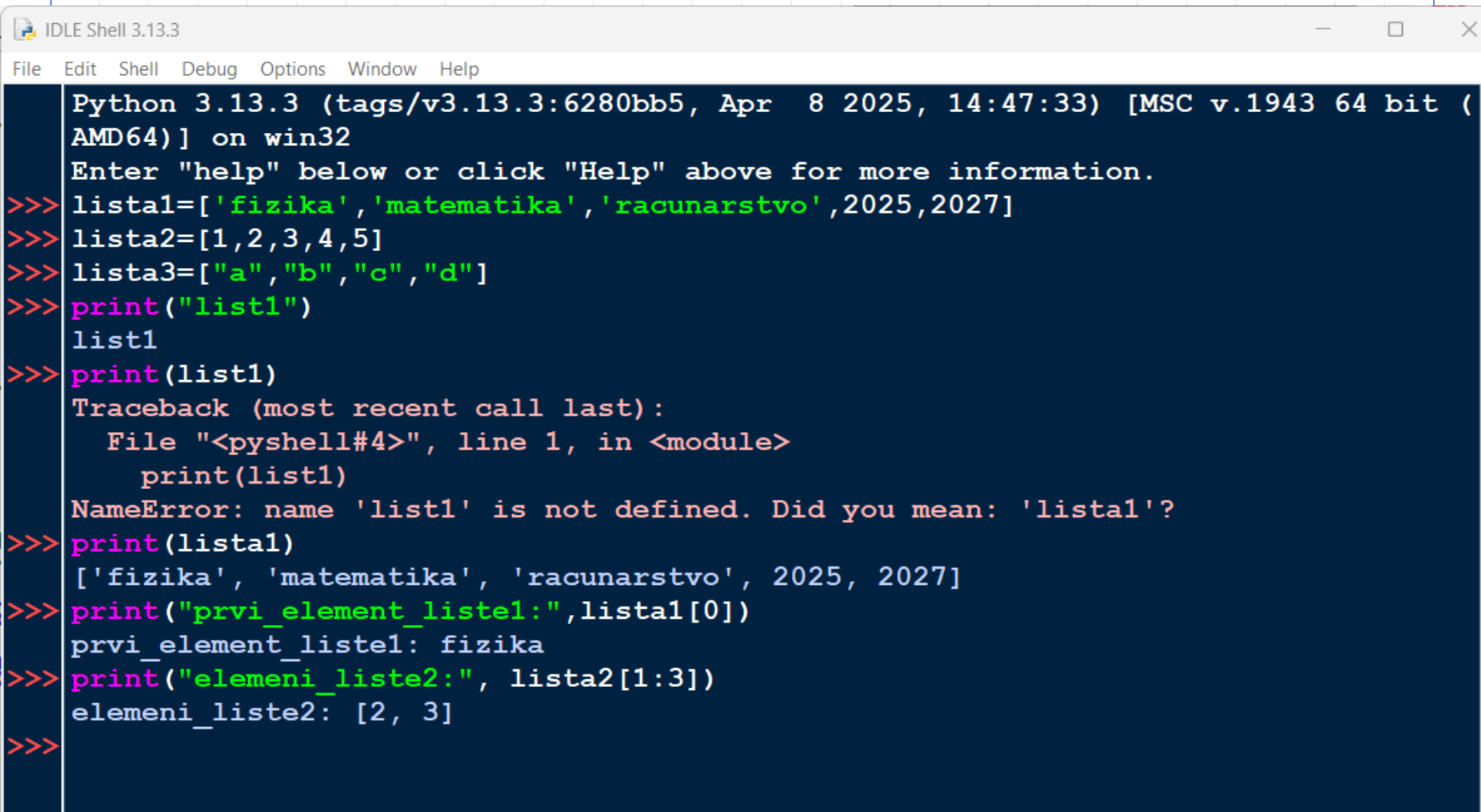
```
fruits = ['banana', 'orange', 'mango', 'lemon']
fruits.sort()
print(fruits)          # sorted in alphabetical order, ['banana', 'lemon', 'mango', 'orange']
fruits.sort(reverse=True)
print(fruits) # ['orange', 'mango', 'lemon', 'banana']
ages = [22, 19, 24, 25, 26, 24, 25, 24]
ages.sort()
print(ages) # [19, 22, 24, 24, 24, 25, 25, 26]

ages.sort(reverse=True)
print(ages) # [26, 25, 25, 24, 24, 24, 22, 19]
```

```
fruits = ['banana', 'orange', 'mango', 'lemon']
print(sorted(fruits)) # ['banana', 'lemon', 'mango', 'orange']
# Reverse order
fruits = ['banana', 'orange', 'mango', 'lemon']
fruits = sorted(fruits, reverse=True)
print(fruits) # ['orange', 'mango', 'lemon', 'banana']
```

sorted(): vraća
uređenu listu bez
modifikovanja
originalne liste

Liste - primeri



```
Python 3.13.3 (tags/v3.13.3:6280bb5, Apr 8 2025, 14:47:33) [MSC v.1943 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>> lista1=['fizika', 'matematika', 'racunarstvo', 2025, 2027]
>>> lista2=[1, 2, 3, 4, 5]
>>> lista3=["a", "b", "c", "d"]
>>> print("list1")
list1
>>> print(list1)
Traceback (most recent call last):
  File "<pyshell#4>", line 1, in <module>
    print(list1)
NameError: name 'list1' is not defined. Did you mean: 'lista1'?
>>> print(lista1)
['fizika', 'matematika', 'racunarstvo', 2025, 2027]
>>> print("prvi_element_liste1:", lista1[0])
prvi_element_liste1: fizika
>>> print("elemeni_liste2:", lista2[1:3])
elemeni_liste2: [2, 3]
>>>
```

Liste - primeri

```
IDLE Shell 3.13.3
File Edit Shell Debug Options Window Help
Python 3.13.3 (tags/v3.13.3:6280bb5, Apr 8 2025, 14:47:33) [MSC v.1943 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>> aList=[123,'xyz','python','abc']
>>> aList.append(2025) # dodavanje novog elementa liste 1
>>> print("Nova lista:", aList)
Nova lista: [123, 'xyz', 'python', 'abc', 2025]
>>> aList=[123,'xyz','python','abc',123]
>>> print("Broj za 123:", aList.count(123)) #Prebrojavanje elemenata liste 2
Broj za 123: 2
>>> print("Broj za xyz:", aList.count(xyz))
Traceback (most recent call last):
  File "<pyshell#5>", line 1, in <module>
    print("Broj za xyz:", aList.count(xyz))
NameError: name 'xyz' is not defined
>>> print("Broj za 123:", aList.count('123'))
Broj za 123: 0
>>> bList=[2026,'python_3']
>>> aList.extend(bList) #prosirenje liste novom 3
>>> print("Prosirena lista aList je:", aList)
Prosirena lista aList je: [123, 'xyz', 'python', 'abc', 123, 2026, 'python_3']
>>> print("Index za xyz je:",aList.index('xzy')) #Odredjivanje indexa elementa liste 4
Traceback (most recent call last):
  File "<pyshell#10>", line 1, in <module>
    print("Index za xyz je:",aList.index('xzy')) #Odredjivanje indexa elementa liste
ValueError: 'xzy' is not in list
>>> print("Index za xyz je:",aList.index('xyz'))
Index za xyz je: 1
>>>
```

Liste - primeri

```
IDLE Shell 3.13.3
File Edit Shell Debug Options Window Help
Python 3.13.3 (tags/v3.13.3:6280bb5, Apr 8 2025, 14:47:33) [MSC v.1943 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>> aList=[123,'xzy','python_3','abc',2025]
>>> aList.insert(3,2020)
>>> print("Nova lista:", aList)
Nova lista: [123, 'xzy', 'python_3', 2020, 'abc', 2025]
>>> print("Lista 1:", aList.pop()) #Uklanjanje poslednjeg elementa liste 1
Lista 1: 2025
>>> print(aList)
[123, 'xzy', 'python_3', 2020, 'abc']
>>> print("Lista 2:", aList.pop(2)) #Uklanjanje elementa sa indexom 2 2
Lista 2: python_3
>>> print(aList)
[123, 'xzy', 2020, 'abc']
>>> aList.remove('xyz') #Brisanje elementa liste 3
Traceback (most recent call last):
  File "<pyshell#7>", line 1, in <module>
    aList.remove('xyz') #Brisanje elementa liste
ValueError: list.remove(x): x not in list
>>> aList.remove('xzy')
>>> print(aList)
[123, 2020, 'abc']
>>> aList.reverse() ##Obrnut redosled elemenata liste 4
>>> print(aList)
['abc', 2020, 123]
>>> aList.sort() #Sortiranje liste 5
Traceback (most recent call last):
  File "<pyshell#12>", line 1, in <module>
    aList.sort() #Sortiranje liste
TypeError: '<' not supported between instances of 'int' and 'str'
>>>
```

Liste - primeri

- Osnovne operacije sa listom:

Python izraz	Rezultat	Opis
<code>len([1, 2, 3])</code>	3	Broj elemenata liste
<code>[1, 2, 3] + [4, 5, 6]</code>	<code>[1, 2, 3, 4, 5, 6]</code>	Konkatenacija
<code>['Hi!'] * 4</code>	<code>['Hi!', 'Hi!', 'Hi!', 'Hi!']</code>	Ponavljjanje
<code>3 in [1, 2, 3]</code>	True	Pripadnost
<code>for x in [1, 2, 3]: print (x, end=' ')</code>	1 2 3	Iteracija

- Indeksiranje i odsecanje liste:

```
L = ['spam', 'Spam', 'SPAM!'];
```

Python izraz	Rezultat	Opis
<code>L[2]</code>	<code>'SPAM!'</code>	Offset ide od nule
<code>L[-2]</code>	<code>'Spam'</code>	Negativan indeks se broji od kraja liste
<code>L[1:]</code>	<code>['Spam', 'SPAM!']</code>	Odsečak od indeksa 1 do kraja liste

3.3 Skup

- Python ima neuređenu kolekciju elemenata koja se zove **SKUP** (set)
- Struktura skupa je slična matematičkim skupovima jer za nju važe unija, presek i razlika
- Skupovi su poput listi u kojima nije dozvoljeno ponavljanje elemenata (ne dozvoljavaju se duplikati!!!)
- Skupovi se definišu vitičastim zagradama **{ }**

```
>>> S = {1,2,3,4,5}
```

- Prazan skup dobijamo konstruktorom `set()`

```
>>> S=set()
```

!!! Notacija `S={}` označava prazan rečnik, a ne skup!

- Skup se može se formirati od sekvence (string, lista), pri čemu se duplikati izbacuju

Skup

- Funkcija `set()` se može koristiti za konverziju objekata u skupove
- Skup možemo definisati na različite načine, npr. `for` petljom

```
IDLE Shell 3.13.3
File Edit Shell Debug Options Window Help
Python 3.13.3 (tags/v3.13.3:6280bb5, Apr 8 2025, 14:47:33) [MSC v.1943 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>> slova=set('abrakadabra')
>>> slova
{'b', 'r', 'd', 'a', 'k'}
>>> studenti=set(['Ana', 'Todor', 'Marko', 'Ana', 'Milan', 'Sofija', 'Marko'])
>>> studenti
{'Sofija', 'Milan', 'Marko', 'Todor', 'Ana'}
>>> brojevi=set([1,4,5,4,4,5,1,2,1,1,2,3,6,3])
>>> brojevi
{1, 2, 3, 4, 5, 6}
>>> string4=set('this is a test')
>>> string4
{'t', 'e', ' ', 'h', 'i', 's', 'a'}
>>> skup5={i**2 for i in range(12)}
>>> skup5
{0, 1, 64, 121, 4, 36, 100, 9, 16, 49, 81, 25}
>>> |
```

Skup

- Python smešta podatke u skup u proizvoljnom redosledu, i ne uvek u redosledu koji smo postavili
- Kod skupova nije važan redosled, već samo podaci koji ga čine
- Indeksiranje kod skupova nema smisla, pa se zato ne može tražiti npr. `s[0]` za set `s`
- Matematički operatori za rad sa skupovima:

Operator	Opis	Primer
	Unija	$\{1,2,3\} \cup \{3,4\} \rightarrow \{1,2,3,4\}$
&	Presek	$\{1,2,3\} \cap \{3,4\} \rightarrow \{3\}$
-	Razlika	$\{1,2,3\} - \{3,4\} \rightarrow \{1,2\}$
^	Simetrična razlika	$\{1,2,3\} \Delta \{3,4\} \rightarrow \{1,2,4\}$
in	Jeste u skupu	$3 \in \{1,2,3\} \rightarrow \text{True}$

Skup - operatori

- Nad skupovima se primenjuju relacijski operatori

Operator	Metoda	Opis
A == B		Vraća True ako je skup A jednak B
A != B		Vraća True ako je skup A različit od B
A <= B	A.issubset(B)	Vraća True ako je A podskup od B
A >= B	A.issuperset(B)	Vraća True ako je A nadskup od B
A < B		Vraća True ako je A pravi podskup od B
A > B		Vraća True ako je A pravi nadskup od B
	A.isdisjoint(B)	Vraća True ako je presek A i B prazan

Skup - metode

- Nad skupovima se primenjuju metode

Metoda	Opis
<code>A.add(element)</code>	Ubacuje element u skup A
<code>A.discard(element)</code>	Uklanja element iz skupa A
<code>A.remove(element)</code>	Uklanja element iz skupa A (javlja grešku ako nije u skupu)
<code>A.clear()</code>	Uklanja sve elemente iz skupa
<code>A.pop()</code>	Vraća slučajno odabran element skupa i uklanja ga
<code>A.copy()</code>	Vraća kopiju skupa

- Napomena:
 - Dodela vrednosti ne pravi kopiju skupa
 - Kada se uradi `A=B`, ubacivanje novog elementa u jedan od skupova ubacuje ga i u drugi, ali kod `A =B.copy()` ne!

Skup - metode

- Broj elemenata skupa (**len**):

```
fruits = {'banana', 'orange', 'mango', 'lemon'}  
len(fruits)
```

- Dodavanje elemenata skupa (**add**, **update**):

```
fruits = {'banana', 'orange', 'mango', 'lemon'}  
fruits.add('lime')
```

```
fruits = {'banana', 'orange', 'mango', 'lemon'}  
vegetables = ('tomato', 'potato', 'cabbage', 'onion', 'carrot')  
fruits.update(vegetables)
```

- Konverzija liste u skup:

```
fruits = ['banana', 'orange', 'mango', 'lemon', 'orange', 'banana']  
fruits = set(fruits) # {'mango', 'lemon', 'banana', 'orange'}
```

- Spajanje skupova (**union**):

```
fruits = {'banana', 'orange', 'mango', 'lemon'}  
vegetables = {'tomato', 'potato', 'cabbage', 'onion', 'carrot'}  
print(fruits.union(vegetables)) # {'lemon', 'carrot', 'tomato', 'banana', 'mango', 'orange', 'cabbage', 'potato', 'onion'}
```

Skup – primeri

```
IDLE Shell 3.13.3
File Edit Shell Debug Options Window Help
Python 3.13.3 (tags/v3.13.3:6280bb5, Apr 8 2025, 14:47:33) [MSC v.1943 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>> {'crvena', 'plava'} == {'plava', 'crvena'}
True
>>> a, b, c = {1, 2, 3, 4, 5}, {3, 4, 5}, {1, 2}
>>> c.add(3) #ubacivanje elementa u skup c
>>> c
{1, 2, 3}
>>> c.discard(3) #uklanjanje elementa iz skupa c
>>> c
{1, 2}
>>> a & b #presek skupova
{3, 4, 5}
>>> b & c
set()
>>> b | c #uniija skupova
{1, 2, 3, 4, 5}
>>> a - b #razlika skupova
{1, 2}
>>> a >= b #a nadskup b
True
>>> b <= a #b podskup a
True
>>> a.clear() #uklanja sve elemente iz skupa
>>> a
set()
>>>
```

3.4. Rečnik

- Python **REČNIK** (dictionary) ima neuređenu kolekciju elemenata koju čine parovi ključ/vrednost (*key/value*) i koja je promenljiva (mutable)
- Rečnik je forma heš-tabele i koristi se za čuvanje parova objekata ključ/vrednost
- Ključevi moraju biti hashable objekti jer se na osnovu njih dobijanje vrednosti elemenata rečnika
- Rečnik je neuređen i pri ispisu redosled elemenata može biti drugačiji nego pri generisanju
- Pri navođenju se ključ i vrednost u paru razdvajaju znakom **:**
- Parovi se navode i razdvajaju zarezom **,**
- Primer:

```
>>>recnik = {'Ime':'Milan','Godine':20,'Smer':'IT'}
```

Rečnik

- U rečnicima ključ ima sličnu ulogu kao indeks
- Ključevi su često stringovi, mada mogu biti celi ili realni brojevi
- U istom rečniku mogu se naći ključevi različitog tipa
- Prazan rečnik je `{ }`, što je analogno sa `[ ]` za praznu listu, ili `' '` za prazan string
- Redosled elemenata u rečniku ne mora biti isti kao redosled dodavanja elemenata u rečnik
- Python menja redosled elemente rečnika u cilju optimizacije performansi

```
recnik = {'Godine':20,'Smer':'IT'}
```

ključ

vrednost

```
recnik['Godine'] = 20
```

Rečnik - osnovne operacije

- Prazan rečnik R može se dobiti na dva načina:

```
>>> A={}
>>> A=dict()
```
- Kreiranje parova moguće je eksplicitno dodelom vrednosti

```
recnik[kljuc]=vrednost
```

```
>>> recnik['Fak']='ETF'
>>> recnik['Ime']='Ana'
>>> recnik
{'Ime': 'Ana', 'Godine': 21, 'Smer': 'IT', 'Fak': 'ETF'}
```
- Promena vrednosti:

```
>>> recnik['Godine']=23
```
- Brisanje elementa:

```
>>> del recnik['Smer']
```
- Brisanje svih ključeva:

```
>>> recnik.clear()
```

Rečnik - metode

Metoda	Opis
<code>A.clear()</code>	Uklanjanje svih elemenata iz rečnika
<code>A.copy()</code>	Vraća plitku (<i>shallow</i>) kopiju rečnika.
<code>dict.fromkeys(seq[, v])</code>	Vraća novi rečnik sa ključevima iz <i>seq</i> i vrednostima <i>v</i> (ako vrednost nije data onda <i>None</i>)
<code>A.get(key[, d])</code>	Vraća vrednost za ključ <i>key</i> . Ako ne postoji , onda vraća <i>d</i> , a ako nije dato <i>d</i> vraća <i>None</i>
<code>A.items()</code>	Vraća <i>view</i> svih elemenata u rečniku u obliku (<i>key, value</i>).
<code>A.keys()</code>	Vraća <i>view</i> svih ključeva u rečniku
<code>A.pop(key[, d])</code>	Uklanja element sa ključem <i>key</i> i vraća njegovu vrednost, ili <i>d</i> ako nije pronađen, a ako je bez <i>d</i> javlja <i>KeyError</i> .
<code>A.popitem()</code>	Uklanja i vraća jedan proizvoljni element (<i>key, value</i>). Javlja <i>KeyError</i> za prazan rečnik

Rečnik - metode

Metoda	Opis
A.setdefault (key[, d])	Ako je <i>key</i> u rečniku, vraća vrednost. Ako nije, dodaje <i>key</i> sa vrednošću <i>d</i> i vraća tu vrednost
A.update ([other])	Ažurira rečnik parovima <i>key/value</i> iz <i>other</i> , ako neki ključ postoji, upisuje preko postojećeg
A.values ()	Vraća <i>view</i> svih vrednosti u rečniku
A.all ()	Vraća True ako su svi ključevi true, ili ako je rečnik prazan
A.any ()	Vraća True ako je bilo koji ključ true. Za prazan vraća False.
len (A)	Vraća broj elemenata u rečniku
sorted (A)	Vraća novu,sortiranu listu ključeva u rečniku

Rečnik - primeri

ključ mora
biti
neizmenljiv

```
IDLE Shell 3.13.3
File Edit Shell Debug Options Window Help
Python 3.13.3 (tags/v3.13.3:6280bb5, Apr 8 2025, 14:47:33) [MSC v.1943 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>> dict={'Ime':'Ana','Godine':23,'God_studija':'Trecal'}
>>> print("dict['Ime']:", dict['Ime'])
dict['Ime']: Ana
>>> print("dict['Godine']:", dict['Godine'])
dict['Godine']: 23
>>> print("dict['Godina_studija']:", dict['godstudija'])
Traceback (most recent call last):
  File "<pyshell#3>", line 1, in <module>
    print("dict['Godina_studija']:", dict['godstudija'])
KeyError: 'godstudija'
>>> dict={'Prezime':'Milic','Godine':25} #Azuriranje elemenata rečenika
>>> print(dict)
{'Prezime': 'Milic', 'Godine': 25}
>>> print("dict['Godine']:", dict['Godine'])
dict['Godine']: 25
>>> del dict['Prezime']
>>> print("dict['Prezime']:", dict['Prezime'])
Traceback (most recent call last):
  File "<pyshell#8>", line 1, in <module>
    print("dict['Prezime']:", dict['Prezime'])
KeyError: 'Prezime'
>>> dict=[['Ime']:'Ana','Godine':23,'God_studija':'Trecal'}
Traceback (most recent call last):
  File "<pyshell#9>", line 1, in <module>
    dict=[['Ime']:'Ana','Godine':23,'God_studija':'Trecal'}
TypeError: unhashable type: 'list'
>>>
```

Rečnik - osnovne operacije

- Poređenje rečnika:

```
>>> dict1={'Ime':'Ana','Godine':25}
>>> dict2={'Ime':'Anka','Godine':22}
>>> dict1={'Ime':'Ana','Godine':25}
>>> print (dict1==dict2) #False
>>> print (dict1==dict3) #True
```

- Kopiranje rečnika:

```
>>> dict1={'Ime':'Ana','Godine':25}
>>> dict4=dict1.copy()
>>> print(dict4) #{'Godine':25,'Ime':'Ana'}
```

- Prikaz svih elemenata rečnika u formi ključ/vrednost:

```
>>> dict1={'Ime':'Ana','Godine':25}
>>> print("Vrednosti: %s" % dict.items())
#Vrednosti: [('Godine',25),('Ime','Ana')]
```

3.5. Torka

- **TORKA**, n-torka (tuple) je sekvenca neizmenljivih objekata
- Za razliku od liste, ne može da se menja po formiranju
- Može da sadrži elemente različitog tipa i često su to elementi iz istog logičkog konteksta, odnosno objekta
- Indeksiranje i segmentacija su isti kao i kod liste
- Prilikom navođenja elemenata torke, oni se odvajaju zarezima
- Navođenje može biti u zagradama ili bez zagrada
- Ako torka sadrži samo jedan element, iza njega mora da stoji zarez npr. ('Python',)
- Kao i kod liste, funkcijom `len` može se dobiti dužina torke, a one imaju kao i liste `count` i `index` metode
- Pošto su torke nepromenljive nemaju neke od metoda koje imaju liste, kao što su `sort` ili `reverse`

Torka

- Primeri torki:

```
>>> torka1=('Matematika','Racunarstvo',3,2025)
```

```
>>> torka2=(1,2,3,4,5,6,7)
```

```
>>> torka3=('Matematika','Racunarstvo',3,2025)
```

```
>>> torka4="a","b","c","d"
```

```
>>> torka5=() #prazna torka
```

```
>>> torka6=(1234,) #torka sa jednim elementom
```

- Ne mogu se menjati vrednosti članova torke!!!

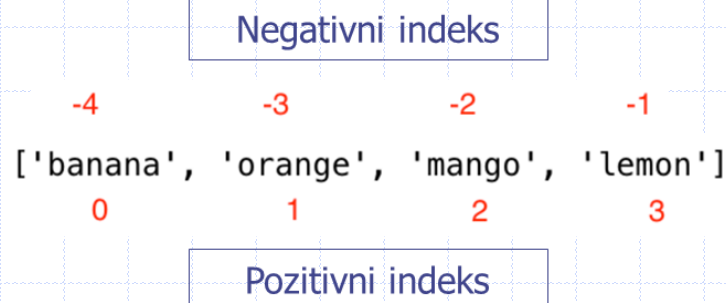
```
>>> torka1=(12,20.25)
>>> torka2=('abc','xyz')
>>> torka1[0]=50
Traceback (most recent call last):
  File "<pyshell#12>", line 1, in <module>
    torka1[0]=50
TypeError: 'tuple' object does not support item assignment
>>> |
```

Torka - metode

- Pristup elementima torke preko indeksa:

```
fruits = ('banana', 'orange', 'mango', 'lemon')
first_fruit = fruits[0]
second_fruit = fruits[1]
last_index = len(fruits) - 1
last_fruit = fruits[last_index]
```

```
first_fruit = fruits[-4]
second_fruit = fruits[-3]
last_fruit = fruits[-1]
```



- Slicing tuples:

```
fruits = ('banana', 'orange', 'mango', 'lemon')
all_fruits = fruits[0:4]    # all items
all_fruits = fruits[0:]     # all items
orange_mango = fruits[1:3]  # doesn't include item at index 3
orange_to_the_rest = fruits[1:]
```

```
fruits = ('banana', 'orange', 'mango', 'lemon')
all_fruits = fruits[-4:]    # all items
orange_mango = fruits[-3:-1] # doesn't include item at index 3
orange_to_the_rest = fruits[-3:]
```

Torka

- Osnovne operacije sa torkama obuhvataju: spajanje, vraćanje dužine torke, ponavljanje torke, provera pripadnosti elementa, iteracija po elementima torke itd...

```
>>> torka1=(12,20.25)
>>> torka2=('abc','xyz')
>>> torka1[0]=50
Traceback (most recent call last):
  File "<pyshell#12>", line 1, in <module>
    torka1[0]=50
TypeError: 'tuple' object does not support item assignment
>>> torka3=torka1+torka2
>>> print(torka3)
(12, 20.25, 'abc', 'xyz')
```

Konkatenacija (spajanje)

- Provera postojanja elementa u torki (**in**):

```
fruits = ('banana', 'orange', 'mango', 'lemon')
print('orange' in fruits) # True
print('apple' in fruits) # False
fruits[0] = 'apple' # TypeError: 'tuple' object does not support item assignment
```

Torka

- Indeksiranje i odsecanje torke:

```
L = ('spam', 'Spam', 'SPAM!');
```

Python izraz	Rezultat	Opis
L[2]	'SPAM!'	Ofset ide od nule
L[-2]	'Spam'	Negativan indeks se broji od kraja n-torke
L[1:]	('Spam', 'SPAM!')	Odsečak od indeksa 1 do kraja n-torke

- Poređenje torke:

```
>>> tuple1,tuple2=(123,'xyz'),(456,'abc')
>>> print((tuple1>tuple2)-(tuple1<tuple2))
-1
>>> print((tuple2>tuple1)-(tuple2<tuple1))
1
>>> tuple3=tuple2+(768,)
>>> print((tuple2>tuple3)-(tuple2<tuple3))
-1
>>> print(len(tuple2))
2
```

Dužina torke

Poređenje elemenata dve torke:

- ako su elementi istog tipa vrši se poređenje i vraća se rezultat
- ako su elementi različitih tipova proverava se da li su brojevi, ako jesu, svode se na isti tip, porede i vraća se rezultat
- ako je jedan element broj drugi element je veći (brojevi su „najmanji” tip) a ako nisu, elementi se porede alfabetski
- ako se dosegne kraj jedne torke a druge ne, duža torke je veća
- ako se iscrpe obe torke i imaju iste podatke vraća se 0

